



Challenge BinaryGecko

Eugenio D. Deluca

(KarasuRØØT) – Date: 08/10/2025

Index

Level 0 (Intro)	1
Tools:	1
Level 1 (Static Key)	2
Level 2 (Key generation)	7
Level 3 (XOR decryption)	12
Level 4 (Math Operations)	17
Level 5 (Unique Key)	19
Level 6 (License file)	22
Level 7 (Obfuscated code)	26
Final conclusion	38
Scripts	39
Challenge2-BinaryGeckoENG.py	39
Challenge4-BinaryGeckoENG.py	40
Challenge5-BinaryGeckoENG.py	42
Challenge6-BinaryGeckoENG.py	43

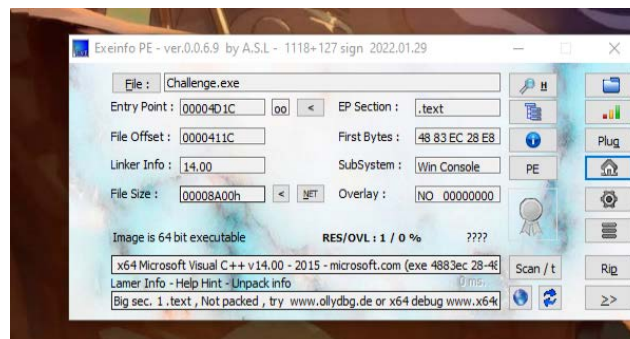
Level 0 (Intro)

This document details the methodology and findings obtained during the resolution of the Reverse Engineering Challenge BinaryGecko '.

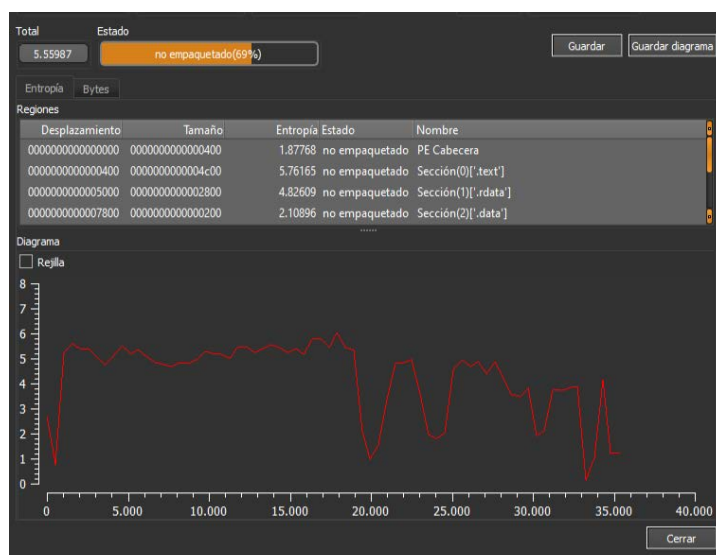
Tools:

- IDA PRO
- .x64
- Wit

As the instructions indicated, I changed the extension from “Challenge.txt” to “Challenge.exe” – Then, out of curiosity, I opened Exeinfo PE – to see what I was up against.



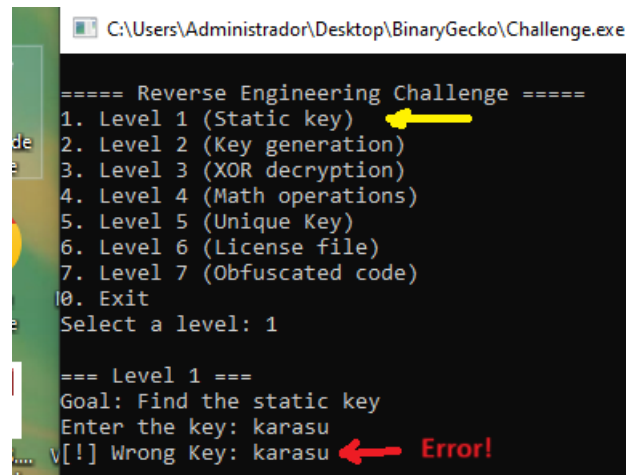
As you can see in that image, it is a 64-bit executable developed in C/C++ (bad news for me since I don't have much experience with 64-bit executables – but it's not a problem).



And very good news for me is not packaged this time.

Level 1 (Static Key)

Well after this little “ intro ” we start with Level 1 (Static key)



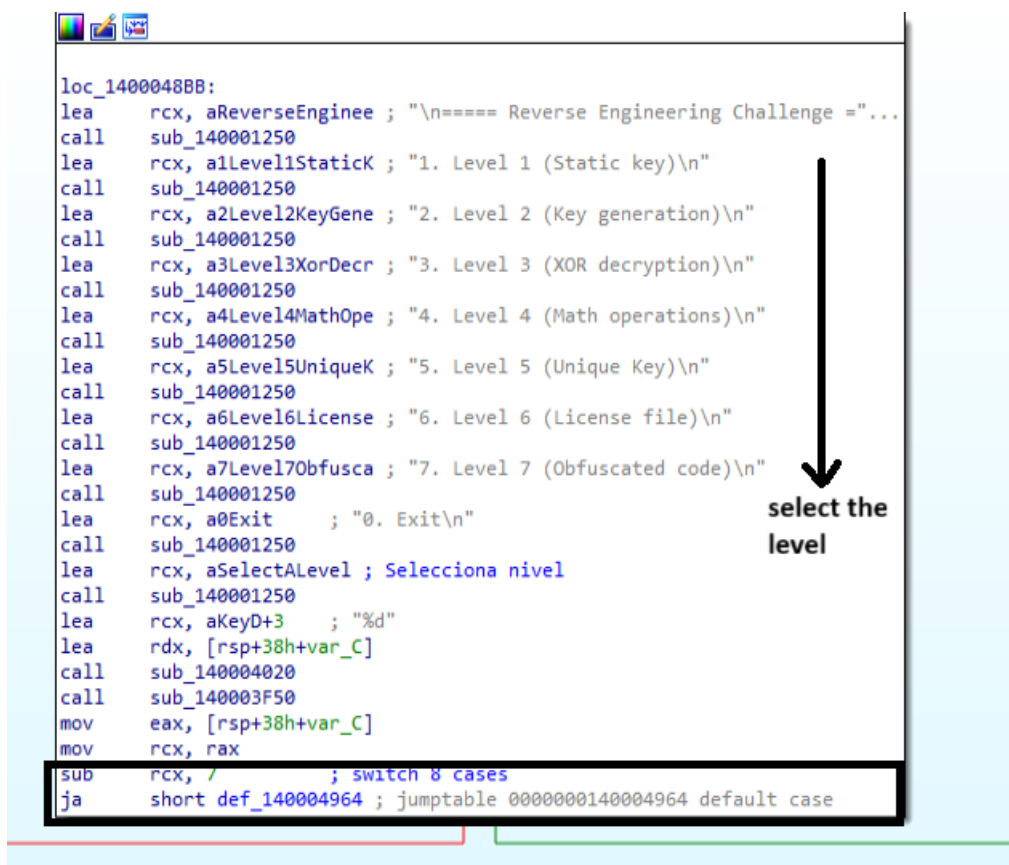
```
C:\Users\Administrador\Desktop\BinaryGecko\Challenge.exe

===== Reverse Engineering Challenge =====
1. Level 1 (Static key)
2. Level 2 (Key generation)
3. Level 3 (XOR decryption)
4. Level 4 (Math operations)
5. Level 5 (Unique Key)
6. Level 6 (License file)
7. Level 7 (Obfuscated code)
0. Exit
Select a level: 1

=== Level 1 ===
Goal: Find the static key
Enter the key: karasu
v[!] Wrong Key: karasu Error!
```

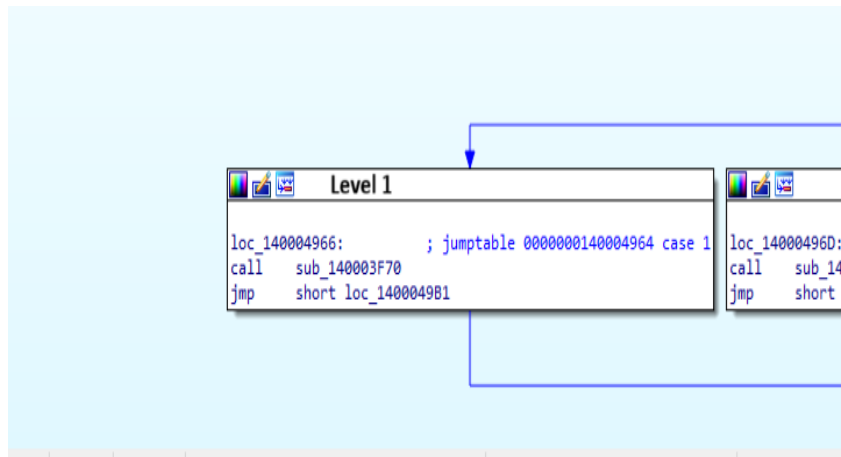
To do a quick test, point to a key as a test (“ karasu ”) and of course, it indicates that you are wrong.

I start by opening IDA PRO:

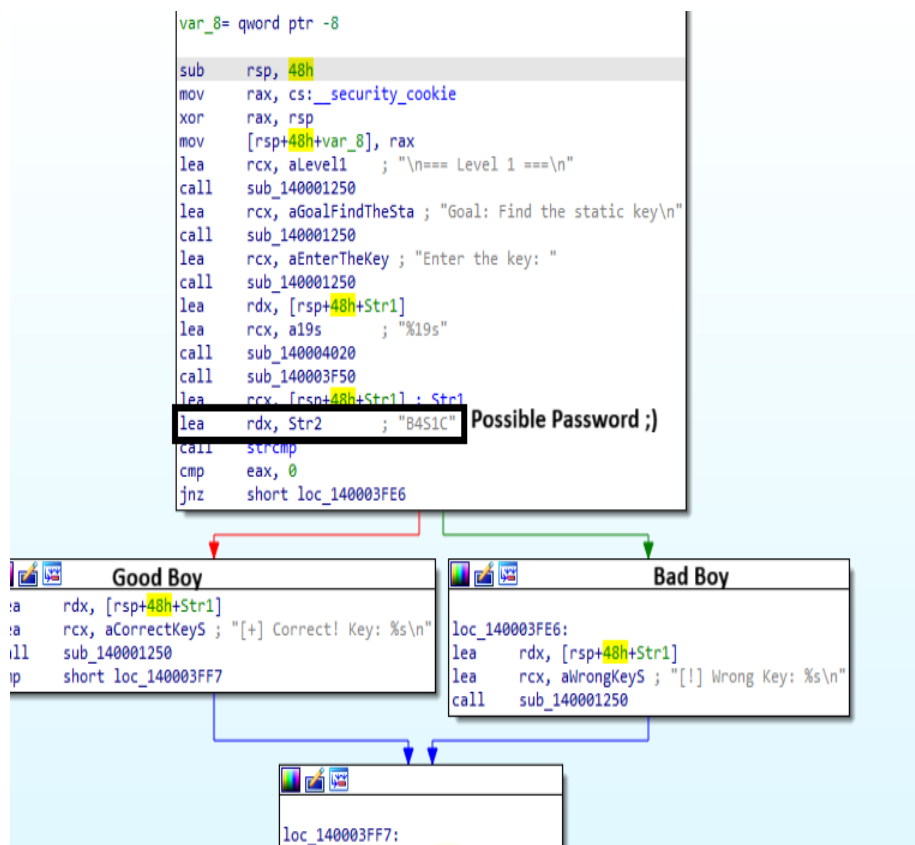


```
loc_1400048BB:
lea rcx, aReverseEngineer ; "\n===== Reverse Engineering Challenge ="...
call sub_140001250
lea rcx, a1Level1StaticK ; "1. Level 1 (Static key)\n"
call sub_140001250
lea rcx, a2Level2KeyGene ; "2. Level 2 (Key generation)\n"
call sub_140001250
lea rcx, a3Level3XorDecr ; "3. Level 3 (XOR decryption)\n"
call sub_140001250
lea rcx, a4Level4MathOpe ; "4. Level 4 (Math operations)\n"
call sub_140001250
lea rcx, a5Level5UniqueK ; "5. Level 5 (Unique Key)\n"
call sub_140001250
lea rcx, a6Level6License ; "6. Level 6 (License file)\n"
call sub_140001250
lea rcx, a7Level7Obfusca ; "7. Level 7 (Obfuscated code)\n"
call sub_140001250
lea rcx, a0Exit ; "0. Exit\n"
call sub_140001250
lea rcx, aSelectAlevel ; Selecciona nivel
call sub_140001250
lea rcx, aKeyD+3 ; "%d"
lea rdx, [rsp+38h+var_C]
call sub_140004020
call sub_140003F50
mov eax, [rsp+38h+var_C]
mov rcx, rax
sub rcx, / ; switch 8 cases
ja short def_140004964 ; jumtable 00000000140004964 default case
```

Of course, this analysis will be performed for the first and only time now. This means that, as we know, it's a program with a "case" statement, which will jump depending on the number we specify.



As we can see, this is the case for all seven difficulty levels of this challenge. For each level, there is a case and then a call to the corresponding function. Therefore, for this first level, I enter " call sub_140003F70."



At this first level, we can clearly differentiate “Good Boy” and “Bad Boy” and additionally, the possible password “ **B4S1C** ” is hardcoded .

00007FF7CA4328D4	lea rcx,qword ptr ds:[7FF7CA4363FF]	00007FF7CA4363FF	"string too long"
00007FF7CA432C9F	lea rdx,qword ptr ds:[7FF7CA4363EA]	00007FF7CA4363EA	"bad array new leng
00007FF7CA432DA6	lea rax,qword ptr ds:[7FF7CA4363A2]	00007FF7CA4363A2	"Unknown exception"
00007FF7CA433E74	lea rcx,qword ptr ds:[7FF7CA4363B4]	00007FF7CA4363B4	"invalid string pos
00007FF7CA433F83	lea rcx,qword ptr ds:[7FF7CA43680E]	00007FF7CA43680E	"n== Level 1 =="
00007FF7CA433F8F	lea rcx,qword ptr ds:[7FF7CA436616]	00007FF7CA436616	"Goal: Find the sta
00007FF7CA433F9B	lea rcx,qword ptr ds:[7FF7CA43650A]	00007FF7CA43650A	"Enter the key: "
00007FF7CA433FAC	lea rcx,qword ptr ds:[7FF7CA436393]	00007FF7CA436393	"%19s"
00007FF7CA433FC2	lea rdx,qword ptr ds:[7FF7CA436442]	00007FF7CA436442	"B451C"
00007FF7CA433FD8	lea rcx,qword ptr ds:[7FF7CA4366F5]	00007FF7CA4366F5	"[+] Correct! Key:
00007FF7CA433FE8	lea rcx,qword ptr ds:[7FF7CA4366E2]	00007FF7CA4366E2	"[!] Wrong Key: %s\
00007FF7CA433FCE	challenge.00007FF7CA433FC2 00007FF7CA436442: "B451C"	00007FF7CA4367FC	"n== Level 2 =="
00007FF7CA433FCE	lea rdx,qword ptr ds:[7FF7CA436442]	00007FF7CA436940	"Goal: Get the corr
00007FF7CA433FCE	call <JMP.&strcmp>	00007FF7CA43656F	"Enter your name: "
00007FF7CA433FCE	cmp eax,0	00007FF7CA436393	"%19s"
00007FF7CA433FCE	jne challenge.7FF7CA433FE6	00007FF7CA43651A	"Enter the generate
00007FF7CA433FCE	lea rdx,qword ptr ss:[rsp+20]	00007FF7CA436393	"%19s"
00007FF7CA433FCE	lea rcx,qword ptr ds:[7FF7CA4366F5]	00007FF7CA4366C2	"[+] Correct! Gener
00007FF7CA433FCE	call challenge.7FF7CA431250	00007FF7CA436740	"[!] wrong key, try
00007FF7CA433FCE	jmp challenge.7FF7CA433FF7	00007FF7CA436160	"\"I"
00007FF7CA433FCE	lea rdx,qword ptr ss:[rsp+20]	00007FF7CA4367EA	"n== Level 3 =="
00007FF7CA433FCE	lea rcx,qword ptr ds:[7FF7CA4366E2]	00007FF7CA436790	"Goal: Key is XOR-e
00007FF7CA433FCE	call challenge.7FF7CA431250	00007FF7CA4364F4	"Enter the final ke
00007FF7CA433FCE	mov rcx,qword ptr ss:[rsp+40]	00007FF7CA436393	"%19s"
00007FF7CA433FCE	xor rcx,rsd	00007FF7CA436670	"real key: %s\n"
00007FF7CA433FCE	cmp rcx,qword ptr ds:[7FF7CA439040]	00007FF7CA43669A	"[+] Congratulator
00007FF7CA433FCE	jne challenge.7FF7CA43400F	00007FF7CA43675A	"[+] Wrong. Keep tr
00007FF7CA433FCE	jmp challenge.7FF7CA43400A	00007FF7CA4367D8	"n== Level 4 =="
00007FF7CA433FCE	add rsp,48	00007FF7CA43663A	"Goal: Calculate th
00007FF7CA433FCE	ret	00007FF7CA436534	"Enter a 4-digit nu
00007FF7CA433FCE	call challenge.7FF7CA4355F0	00007FF7CA43639E	"%4s"
00007FF7CA433FCE	int3	00007FF7CA436422	"KEY%d"
00007FF7CA433FCE	lea rcx,qword ptr ds:[7FF7CA436592]	00007FF7CA436592	"Now enter the calc
00007FF7CA433FCE	lea rcx,qword ptr ds:[7FF7CA436393]	00007FF7CA436393	"%19s"
00007FF7CA433FCE	lea rcx,qword ptr ds:[7FF7CA43667E]	00007FF7CA43667E	"[+] Success! Valid
00007FF7CA433FCE	lea rcx,qword ptr ds:[7FF7CA436740]	00007FF7CA436740	"[!] wrong key, try
00007FF7CA433FCE	lea rcx,qword ptr ds:[7FF7CA436820]	00007FF7CA436820	"n== Level 5 (Uni
00007FF7CA433FCE	lea r8,qword ptr ds:[7FF7CA436438]	00007FF7CA436438	"%s-%s-REV"
00007FF7CA433FCE	lea rcx,qword ptr ds:[7FF7CA4368D8]	00007FF7CA4368D8	"[!] Debugger detec
00007FF7CA433FCE	lea rcx,qword ptr ds:[7FF7CA4365E8]	00007FF7CA4365E8	"Run without de
00007FF7CA433FCE	lea rcx,qword ptr ds:[7FF7CA43650A]	00007FF7CA43650A	"Enter the key: "
00007FF7CA433FCE	lea rcx,qword ptr ds:[7FF7CA436398]	00007FF7CA436398	"%255s"
00007FF7CA433FCE	lea rcx,qword ptr ds:[7FF7CA4366F5]	00007FF7CA4366F5	"[+] Correct! Key:
00007FF7CA433FCE	lea rcx,qword ptr ds:[7FF7CA436708]	00007FF7CA436708	"[+] Wrong. Valid k
00007FF7CA433FCE	lea rdx,qword ptr ds:[7FF7CA436470]	00007FF7CA436470	"BinaryGecko"
00007FF7CA433FCE	lea rdx,qword ptr ds:[7FF7CA436468]	00007FF7CA436468	"9898"
00007FF7CA433FCE	lea rcx,qword ptr ds:[7FF7CA436428]	00007FF7CA436428	"licencia.enc"
00007FF7CA433FCE	lea rdx,qword ptr ds:[7FF7CA436435]	00007FF7CA436435	"rb"
00007FF7CA433FCE	lea rcx,qword ptr ds:[7FF7CA436772]	00007FF7CA436772	"Error: License key

Search:

ntdll.dll 100%

Command: Commands are comma separated (like assembly instructions): mov eax, ebx

Paused 10241 string(s) in 2437ms

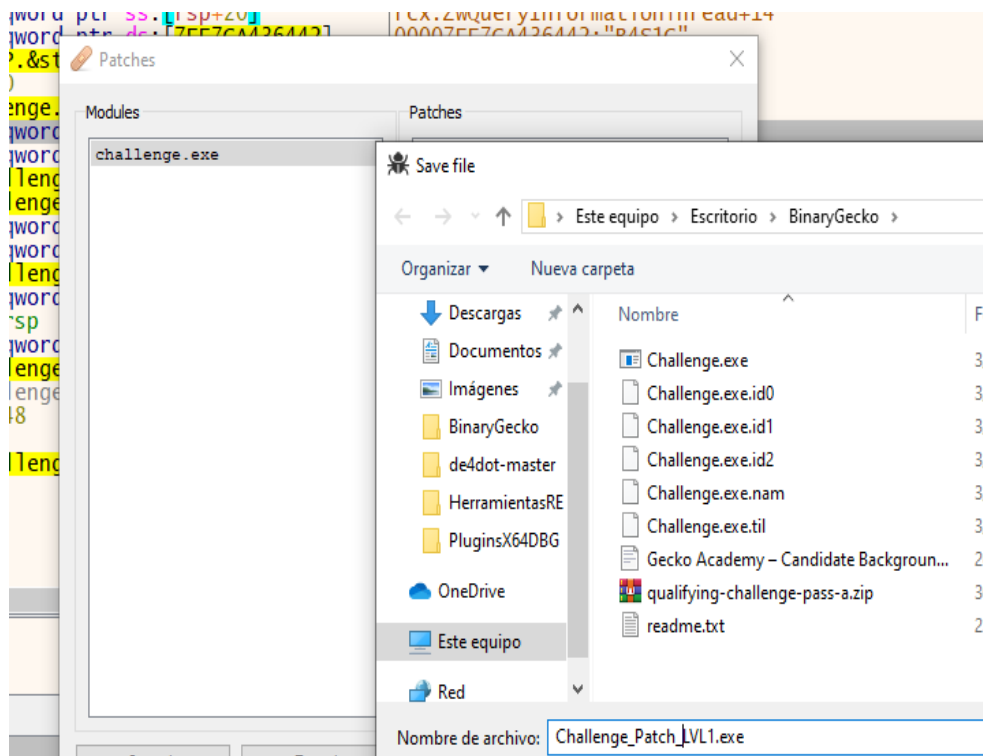
Since the password is hardcoded , the only option would be to patch it.
The line to change is:

00007FF7CA433FCE	83F8 00	cmp eax,0
00007FF7CA433FD1	75 13	jne challenge.7FF7CA433FE6
00007FF7CA433FD3	48:8D5424 20	lea rdx,qword ptr ss:[rsp+20]
00007FF7CA433FD8	48:8D0D 16270000	lea rcx,qword ptr ds:[7FF7CA4366F5]

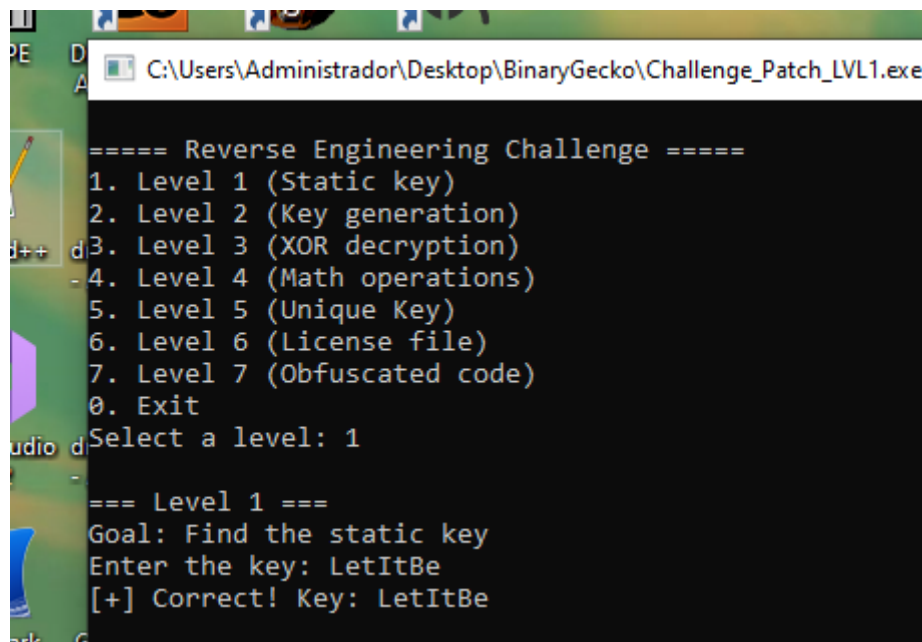
I decided to replace it with:

00007FF7CA433FCE	83F8 00	cmp eax,0
00007FF7CA433FD1	74 13	je challenge.7FF7CA433FE6
00007FF7CA433FD3	48:8D5424 20	lea rdx,qword ptr ss:[rsp+20]

Then I patch the program:

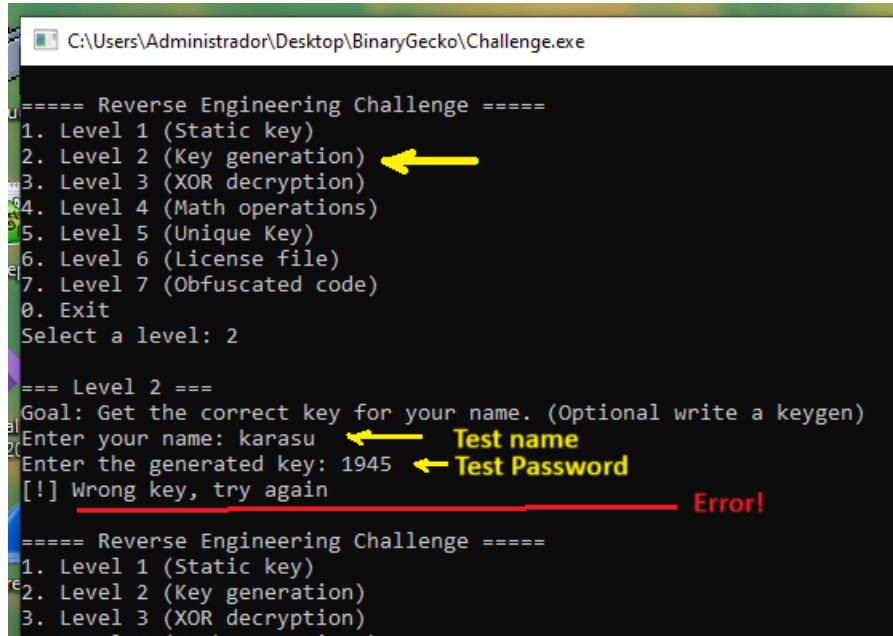


And I run it – I use any password – for example LetItBe The file “Challenge_Patch_LVL1.exe” will also be attached.



Level 2 (Key generation)

Let's start with Level 2:



```
C:\Users\Administrador\Desktop\BinaryGecko\Challenge.exe

===== Reverse Engineering Challenge =====
1. Level 1 (Static key)
2. Level 2 (Key generation)
3. Level 3 (XOR decryption)
4. Level 4 (Math operations)
5. Level 5 (Unique Key)
6. Level 6 (License file)
7. Level 7 (Obfuscated code)
0. Exit
Select a level: 2

=== Level 2 ===
Goal: Get the correct key for your name. (Optional write a keygen)
Enter your name: karasu
Enter the generated key: 1945
[!] Wrong key, try again
Error!

===== Reverse Engineering Challenge =====
1. Level 1 (Static key)
2. Level 2 (Key generation)
3. Level 3 (XOR decryption)
4. Level 4 (Math operations)
```



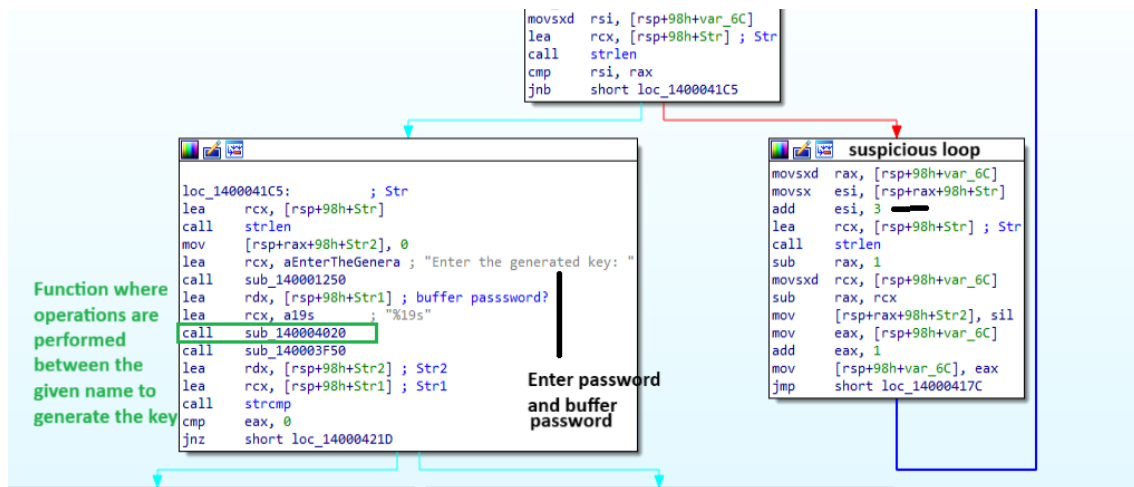
```
sub_140004120 proc near
var_6C= dword ptr -6Ch
Str1= byte ptr -68h
Str2= byte ptr -48h
Str= byte ptr -28h
Name= qword ptr -10h

push    rsi
sub     rsp, 90h
mov     rax, cs:_security_cookie
xor     rax, rsp
mov     [rsp+98h+Name], rax
lea     rcx, aLevel2 ; "\n=== Level 2 ===\n"
call    sub_140001250
lea     rcx, aGoalGetTheCorr ; "Goal: Get the correct key for your name"...
call    sub_140001250
lea     rcx, aEnterYourName ; "Enter your name: "
call    sub_140001250
lea     rdx, [rsp+98h+Str] ; buffer name?
lea     rcx, a19s ; "%19s"
call    sub_140004020
call    sub_140003F50
mov     [rsp+98h+var_6C], 0

loc_14000417C:
movsxd  rsi, [rsp+98h+var_6C]
lea     rcx, [rsp+98h+Str] ; Str
call    strlen
```

As we see in the previous image, there is where we enter the “ name ” and below is the “name buffer” – and in the image that follows, on the one hand, this is where we enter our key and its corresponding buffer, in addition to the Function where operations are performed between the given name to generate the key.

And then there's a loop which, as we see, for example, performs operations, so I assume it's responsible for performing operations with our user.



In order not to waste time, I just place a Breakpoint on the “lea rcx , [rsp+98h+Str1] ; Str1”.

As I did in the previous exercise, I enter a test name (“ karasu ”) and password (“12345”).

```

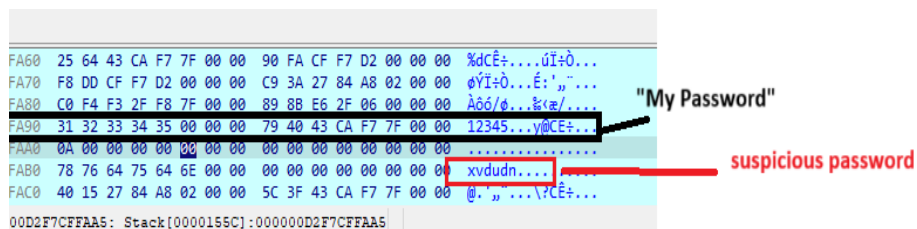
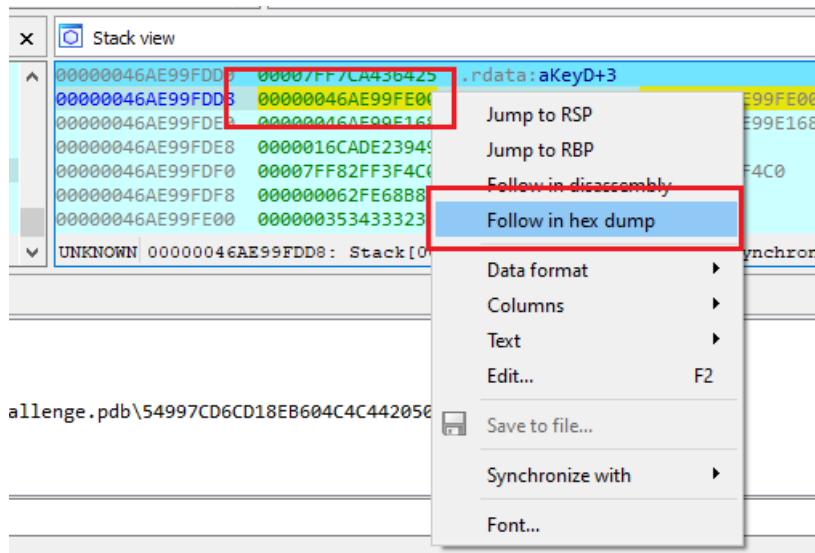
C:\Users\Administrator\Desktop\BinaryGecko\Challenge.exe

===== Reverse Engineering Challenge =====
1. Level 1 (Static key)
2. Level 2 (Key generation)
3. Level 3 (XOR decryption)
4. Level 4 (Math operations)
5. Level 5 (Unique Key)
6. Level 6 (License file)
7. Level 7 (Obfuscated code)
0. Exit
Select a level: 2

=== Level 2 ===
Goal: Get the correct key for your name. (Optional write a keygen)
Enter your name: karasu
Enter the generated key: 12345

```

There the program stops, so I thought I'd check the Stack – Follow in hex dump – on the line “00000046AE99FDD8 00000046AE99FE00”.



Then the program continues and ends up indicating that it is not the correct password, so I decided to try brute force style – then, name (“ karasu ”) and password (“ xvdudn ”) and it is accepted correctly.

```

C:\Users\Administrador\Desktop\BinaryGecko\Challenge.exe

===== Reverse Engineering Challenge =====
1. Level 1 (Static key)
2. Level 2 (Key generation)
3. Level 3 (XOR decryption)
4. Level 4 (Math operations)
5. Level 5 (Unique Key)
6. Level 6 (License file)
7. Level 7 (Obfuscated code)
0. Exit
Select a level: 2

=== Level 2 ===
Goal: Get the correct key for your name. (Optional write a keygen)
Enter your name: karasu
Enter the generated key: xvdudn
[+] Correct! Generated key: xvdudn

===== Reverse Engineering Challenge =====
1. Level 1 (Static key)
2. Level 2 (Key generation)
3. Level 3 (XOR decryption)
4. Level 4 (Math operations)
5. Level 5 (Unique Key)
6. Level 6 (License file)
7. Level 7 (Obfuscated code)
0. Exit
Select a level:

```

Yes , it does, but wait, someone once said to me: "Why don't you try another name?" So I tried it:

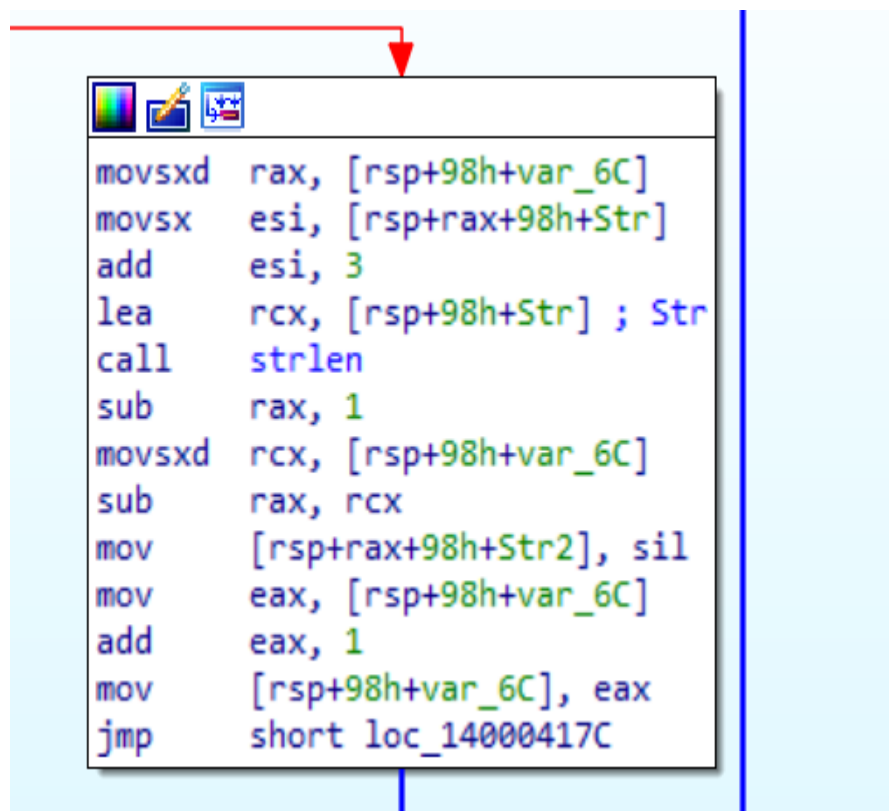
```
===== Reverse Engineering Challenge =====
1. Level 1 (Static key)
2. Level 2 (Key generation)
3. Level 3 (XOR decryption)
4. Level 4 (Math operations)
5. Level 5 (Unique Key)
6. Level 6 (License file)
7. Level 7 (Obfuscated code)
0. Exit
Select a level: 2

=== Level 2 ===
Goal: Get the correct key for your name. (Optional write a keygen)
Enter your name: solid
Enter the generated key: xvdudn
[!] Wrong key, try again

===== Reverse Engineering Challenge =====
1. Level 1 (Static key)
2. Level 2 (Key generation)
3. Level 3 (XOR decryption)
4. Level 4 (Math operations)
5. Level 5 (Unique Key)
6. Level 6 (License file)
7. Level 7 (Obfuscated code)
0. Exit
Select a level: _
```

Of course it didn't work.

After analyzing it carefully, I realized that the answer was right there, and it was the famous loop.



Without a doubt, what the loop does is take the previously typed user string and go through it, adding +3 to each character – then inverting it. Example with karasu →

Input Character	Original ASCII Value	Operation (ASCII+3)	New ASCII Value	Cipher Character
k	107	107+3	110	n
t	97	97+3	100	d
r	114	114+3	117	or
t	97	97+3	100	d
s	115	115+3	118	v
o	117	117+3	120	x

At first, I must admit, I fell into the trap. Wanting to speed things up, I tried the username karasu and the password ndudvx , and logically, I failed. But after a few minutes, I realized I was missing a step: inverting it. That is, the program itself takes each character and then inverts it.

Perform some tests for example:

```

1. Level 1 (Static key)
0. Exit
Select a level: 2

=== Level 2 ===
Goal: Get the correct key for your name. (Optional write a keygen)
Enter your name: solid
Enter the generated key: glorv
[+] Correct! Generated key: glorv

===== Reverse Engineering Challenge =====
1. Level 1 (Static key)

```

```

=== Level 2 ===
Goal: Get the correct key for your name. (Optional write a keygen)
Enter your name: ricardo
Enter the generated key: rgudflu
[+] Correct! Generated key: rgudflu

===== Reverse Engineering Challenge =====
1. Level 1 (Static key)

```

So, my theory was correct. I designed the script, a screenshot of which I'll include here, and I'll attach it as "Challenge2-BinaryGeckoENG.py."

Level 3 (XOR decryption)

Let's start with level 3:

```
===== Reverse Engineering Challenge =====
1. Level 1 (Static key)
2. Level 2 (Key generation)
3. Level 3 (XOR decryption) ←
4. Level 4 (Math operations)
5. Level 5 (Unique Key)
6. Level 6 (License file)
7. Level 7 (Obfuscated code)
10. Exit
Select a level: 3

=== Level 3 ===
Goal: Key is XOR-encrypted
Enter the final key: 123456 ← My Test Password
V
===== Reverse Engineering Challenge =====
```

Well, the first problem I had was that I gave it any password, but it didn't tell me anything else, so I opened IDA and started trying to analyze the situation.

```
sub_140004250 proc near
var_50= dword ptr -50h
var_4C= dword ptr -4Ch
Str2= byte ptr -48h
var_44= byte ptr -44h
Str= byte ptr -28h
var_8= qword ptr -8

sub     rsp, 78h
mov     rax, cs:__security_cookie
xor     rax, rsp
mov     [rsp+78h+var_8], rax
mov     eax, cs:dword_140006160
mov     [rsp+78h+var_4C], eax
lea     rcx, aLevel3      ; "\n=== Level 3 ===\n"
call    sub_140001250
lea     rcx, aGoalKeyIsXorEn ; "Goal: Key is XOR-encrypted\n"
call    sub_140001250
lea     rcx, aEnterTheFinalK ; "Enter the final key: "
call    sub_140001250
lea     rdx, [rsp+78h+Str]
lea     rcx, a19s          ; "%19s"
call    sub_140004020
call    sub_140003F50
lea     rcx, [rsp+78h+Str] ; Str
call    strlen
cmp     rax, 4
jbe     short loc_1400042B9
```

Buy length with 4 → The length of the entered string (key) is calculated and saved in rax

Since I'm just starting out with reverse engineering, there are things I need to review and double-check. And in one of the tests, it appeared, and seeing the previous screen, of course, my first test password didn't work because I tried 6 characters (123456) and this time I tried 4 random ones (z{ xy }):

```

rt      === Level 3 ===
xit_tab Goal: Key is XOR-encrypted
        Enter the final key: z{xy
        real key: X0R3
tion    [-] Wrong. Keep trying
gth(voi

```

Is the X0R3 key the real one? There's no one good enough in this world to tell you the correct key. Forgive me if I doubt these intentions, but I decided to give it a try.

And indeed, in theory that is the key:

```

t_tab  === Level 3 ===
        Goal: Key is XOR-encrypted
on      Enter the final key: X0R3
h(voi   real key: X0R3
de      [+] Congratulations! Decrypted key: X0R3
        ----- Reverse Engineering Challenge -----

```

But since it's a BinaryGecko challenge and I'm already at difficulty level 3, I don't believe it's that easy, so I did some more research. I'll use x64 for dynamic review.

B5	76 02	jbe	challenge.7FF7CA4342B9	
B7	EB 7B	jmp	challenge.7FF7CA434334	
B9	C74424 28 00000000	mov	dword ptr ss:[rsp+28],0	
C1	837C24 28 04	cmp	dword ptr ss:[rsp+28],4	Validate that the password is 4
C6	7D 23	jge	challenge.7FF7CA4342EB	
C8	48:634424 28	movsxd	rax,dword ptr ss:[rsp+28]	
CD	0FBE4404 2C	movsx	eax,byte ptr ss:[rsp+rax+2C]	XOR with 7A
D2	83F0 7A	xor	eax,7A	
D5	48:634C24 28	movsxd	rcx,dword ptr ss:[rsp+28]	
DA	88440C 30	mov	byte ptr ss:[rsp+rcx+30],al	
DE	8B4424 28	mov	eax,dword ptr ss:[rsp+28]	
E2	83C0 01	add	eax,1	
E5	894424 28	mov	dword ptr ss:[rsp+28],eax	
E9	EB D6	jmp	challenge.7FF7CA4342C1	

00007FF7CA4342AC	E8 CF170000	call <JMP.&strlen>
00007FF7CA4342B1	48:83F8 04	cmp rax,4
00007FF7CA4342B5	76 02	jbe challenge.7FF7CA4342B9
00007FF7CA4342B7	EB 7B	jmp challenge.7FF7CA434334
00007FF7CA4342B9	C74424 28 00000000	mov dword ptr ss:[rsp+28],0
00007FF7CA4342C1	837C24 28 04	cmp dword ptr ss:[rsp+28],4
00007FF7CA4342C6	7D 23	jge challenge.7FF7CA4342EB
00007FF7CA4342C8	48:634424 28	movsxd rax,dword ptr ss:[rsp+28]
00007FF7CA4342CD	0FBE4404 2C	movsx eax,byte ptr ss:[rsp+rax+2C]
00007FF7CA4342D2	83F0 7A	xor eax,7A
00007FF7CA4342D5	48:634C24 28	movsxd rcx,dword ptr ss:[rsp+28]
00007FF7CA4342DA	88440C 30	mov byte ptr ss:[rsp+rcx+30],al
00007FF7CA4342DE	8B4424 28	mov eax,dword ptr ss:[rsp+28]
00007FF7CA4342E2	83C0 01	add eax,1
00007FF7CA4342E5	894424 28	mov dword ptr ss:[rsp+28],eax
00007FF7CA4342E9	EB D6	jmp challenge.7FF7CA4342C1
00007FF7CA4342EB	C64424 34 00	mov byte ptr ss:[rsp+34],0

What happens here is that it enters the loop, compares if it doesn't reach 4 and continues, if it exceeds 4 it exits the loop.

The screenshots show the execution of a loop across four iterations (LAP 1 to LAP 4). Each screenshot displays the instruction pointer, assembly code, and a register window. The assembly code is identical to the one in the first screenshot. The register window shows the state of RAX, RBX, RCX, RDX, RSP, RSI, RDI, R8, R9, R10, R11, R12, and R13. The loop counter in RAX increases from 0 to 4 over the four iterations.

And in these screenshots you can see how with each passing lap, it generates the XOR key. Then I decided to review this exercise again. I really thought that it couldn't be the only answer, so I started to investigate a little more.

My hypothesis is that XOR3 is the "decrypted" key, so to speak. So perhaps finding the correct key.

What the program itself does is perform the XOR EAX ,7A operation , so this means that it will do the following:

The first screenshot shows a Windows calculator in Programmer mode. The operation $58 \text{ XOR } 7A = 22$ is displayed. The second screenshot shows a hex editor with a table of characters and their ASCII values. The row for '22' (hex 16) is highlighted, showing the character '2' and its ASCII value 50.

Additionally to justify it, the x64 indicates:

```

eax=0
byte ptr ss:[rsp+rax*1+2C]=[0000007EEF6FA1C "\J(I)"=22 '\']
.text:00007FF7CA4342CD challenge.exe:$42CD #36CD

```

If we follow the instructions:

The first screenshot shows a Windows calculator in Programmer mode. The operation $22 \text{ XOR } 7A = 58$ is displayed. The second screenshot shows a hex editor with a table of characters and their ASCII values. The row for '58' (hex 36) is highlighted, showing the character 'X' and its ASCII value 88.

The third screenshot shows a Windows calculator in Programmer mode. The operation $4A \text{ XOR } 7A = 30$ is displayed. The fourth screenshot shows a hex editor with a table of characters and their ASCII values. The row for '30' (hex 1E) is highlighted, showing the character '0' and its ASCII value 48.

The fifth screenshot shows a Windows calculator in Programmer mode. The operation $28 \text{ XOR } 7A = 52$ is displayed. The sixth screenshot shows a hex editor with a table of characters and their ASCII values. The row for '52' (hex 34) is highlighted, showing the character '4' and its ASCII value 68.

The seventh screenshot shows a Windows calculator in Programmer mode. The operation $49 \text{ XOR } 7A = 33$ is displayed. The eighth screenshot shows a hex editor with a table of characters and their ASCII values. The row for '33' (hex 21) is highlighted, showing the character '3' and its ASCII value 51.

So then:

Assembly code snippet:

```

00007FF7CA4342E5 894424 28 mov dword ptr ss:[rsp+28],eax
00007FF7CA4342E9 EB D6 jmp challenge.7FF7CA4342C1
00007FF7CA4342EB C64424 34 00 mov byte ptr ss:[rsp+34],0
00007FF7CA4342F0 48:8D5424 30 lea rdx,qword ptr ss:[rsp+30]
00007FF7CA4342F5 48:8D0D 74230000 lea rcx,qword ptr ds:[7FF7CA436670]

```

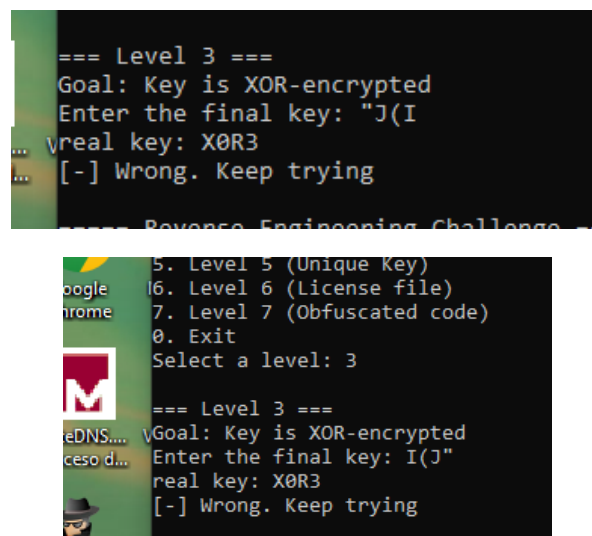
Memory dump (Address | Hex | ASCII):

Address	Hex	ASCII
000000E7EEF6F9F0	C0 F4 F3 2F F8 7F 00 00	A00/0...@u01c...
000000E7EEF6FA00	78 00 00 00 00 00 00 00	x...0000
000000E7EEF6FA10	0A 00 00 00 00 00 00 00	04 00 00 00 22 4A 28 49
000000E7EEF6FA20	58 30 52 33 00 00 00 00	XOR3.
000000E7EEF6FA30	00 1F 39 84 6C 02 00 00	SC 3F 43 CA F7 7F 00 00
000000E7EEF6FA40	31 39 34 35 00 7F 00 00	30 F4 F3 2F F8 7F 00 00
000000E7EEF6FA50	40 F4 F3 2F F8 7F 00 00	60 BF 32 84 6C 02 00 00

Annotations in the image:

- "decrypted" key: J(I
- My Key: XOR3
- correct key: XOR3

So the result should be “J(I – or if it were the inverse – I(J”



In short, the only accepted key is obviously XOR3 –

Level 4 (Math Operations)

```

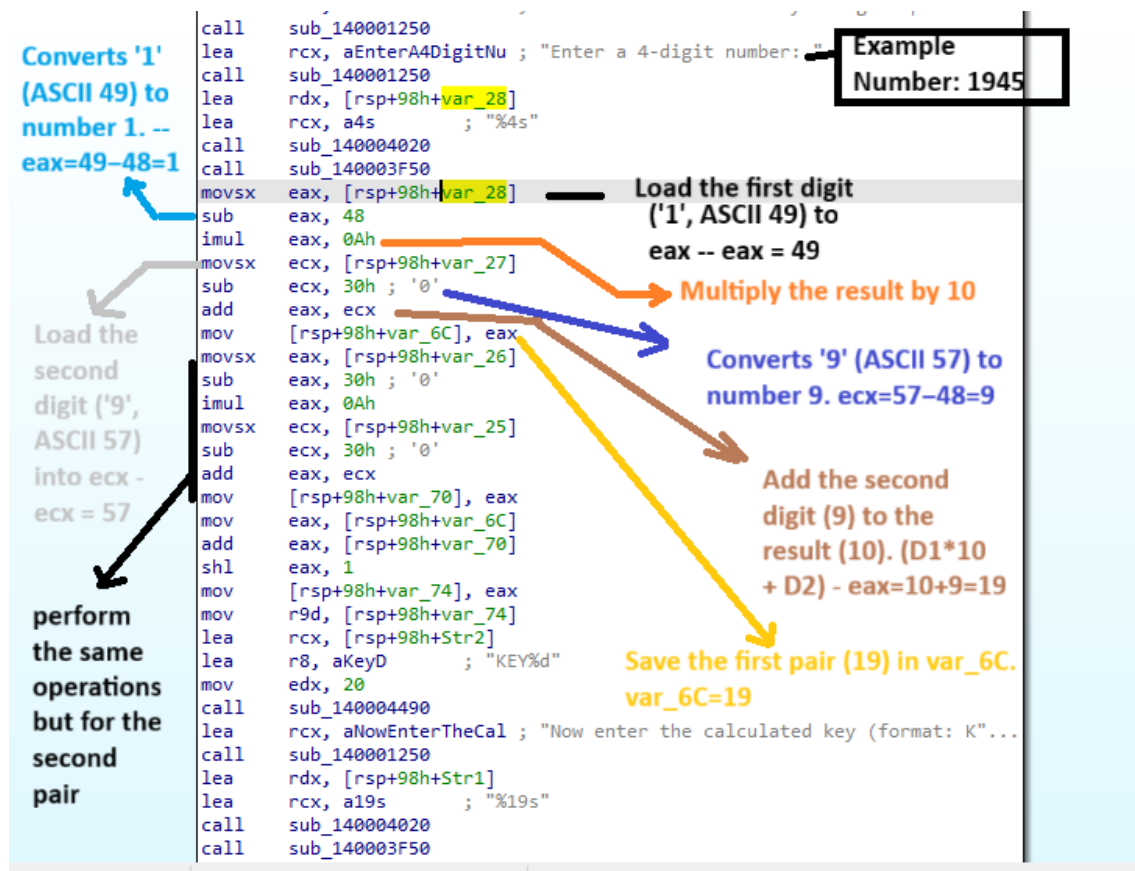
===== Reverse Engineering Challenge =====
1. Level 1 (Static key)
2. Level 2 (Key generation)
3. Level 3 (XOR decryption)
4. Level 4 (Math operations)
5. Level 5 (Unique Key)
6. Level 6 (License file)
7. Level 7 (Obfuscated code)
10. Exit
Select a level: 4

=== Level 4 ===
Goal: Calculate the key using simple math operations
Enter a 4-digit number: 1234
Now enter the calculated key (format: KEYXXXX): KEY1234
[!] Wrong key, try again
  
```

2 Test Password (Format KEYNNNN)

Okay, let's start with the fourth challenge. First, it asks the user to enter four numbers. Then, it asks again for the calculated key in the format "KEYXXXX" -

After analyzing it statically and doing some dynamic testing, I realized it wasn't as problematic as it seemed. It just performs several operations, and that sometimes causes you to get lost.



I apologize in advance for so many colors, but I couldn't think of any other way to explain it better.

But in short what the program does is – Requests a 4 digit number (in our example it will be 1945) – then separates it in two – that is, takes 19 on one side – and 45 on the other – then adds them $\rightarrow 19 + 45 = 64$ – then multiplies it by 2 – $64 \times 2 = 128$ and then concatenates the word KEY – Logically, then compares “KEY128” with the 2nd key that is entered and if they are not equal, it jumps to “bad guy” and ends. If not, it jumps to “good guy” and we solved the exercise.

Although not expressly requested, I took the liberty of making a small keygen – I will attach said script with the name “Challenge4-BinaryGeckoENG.py” referring to this point.

Level 5 (Unique Key)

```
==== Reverse Engineering Challenge ====
1. Level 1 (Static key)
2. Level 2 (Key generation)
3. Level 3 (XOR decryption)
4. Level 4 (Math operations)
5. Level 5 (Unique Key)
6. Level 6 (License file)
7. Level 7 (Obfuscated code)
10. Exit
Select a level: 5

=== Level 5 (Unique Key) ===
Enter the key: 12345
[-] Wrong. Valid key was: DEFC666-Administrador-REV

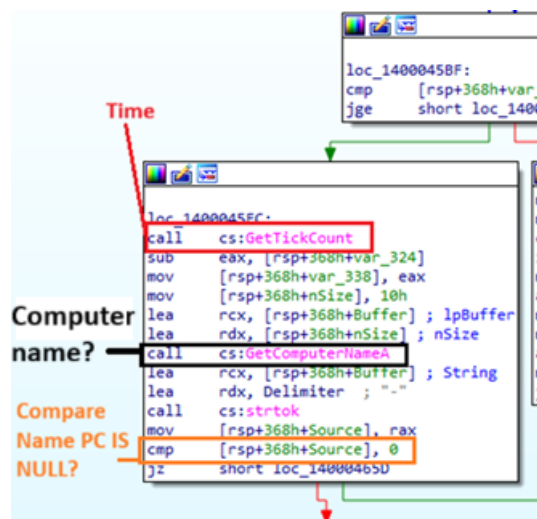
==== Reverse Engineering Challenge ====
1. Level 1 (Static key)
```

Well, starting challenge number 5, they ask me for a password (it doesn't specify the format or length. To my surprise, it not only reports that that is not the password and that the valid password is "DEFC666-Administrator-REV".

I tried a completely different key, but it reports exactly the same thing:

```
=== Level 5 (Unique Key) ===
Enter the key: 56656
[-] Wrong. Valid key was: DEFC666-Administrador-REV
```

When I start it in IDA PRO – and do a quick check, I notice the following:

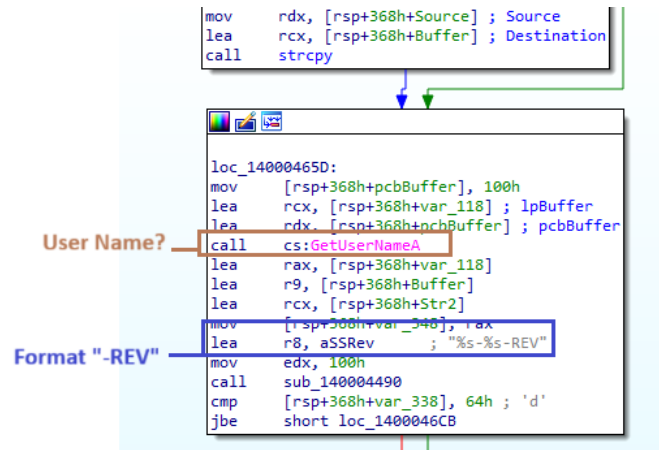


Well, it seems we have some details here that I would like to express.

According to MSDN “ GetTickCount ”: Retrieves the number of milliseconds that have elapsed since the system started, up to 49.7 days.

According to MSDN, “ GetComputerNameA ”: Retrieves the NetBIOS name of the local computer. This name is set at system startup, when the system reads it from the registry.

also compares/checks (at least it's my estimate based on how it is given [RSP+368h+Source], 0 – if a token was found (if the team name was not empty).

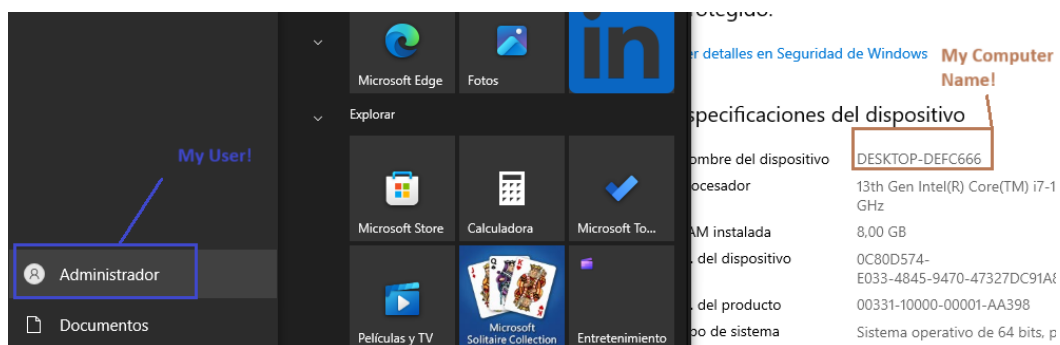


In the image above, it shows us two important things:

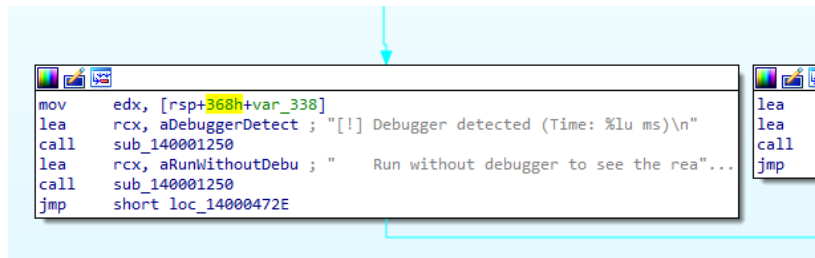
According to MSDN “ GetUserNameA ”: Retrieves the name of the user associated with the current thread.

Also, below prepare to have a -REV format

If we think about it for a minute it makes some sense if we compare it with what the program told us when we tried – it told us that the key was “DEFC666-Administrator-REV” – So it makes sense, since it takes the name of the PC + The name of the user and concatenates “-REV”
In my environment:



Of course, this also has “Good Guy” and “Bad Guy” but it also has a block of code for Antidebugger that is related to time.

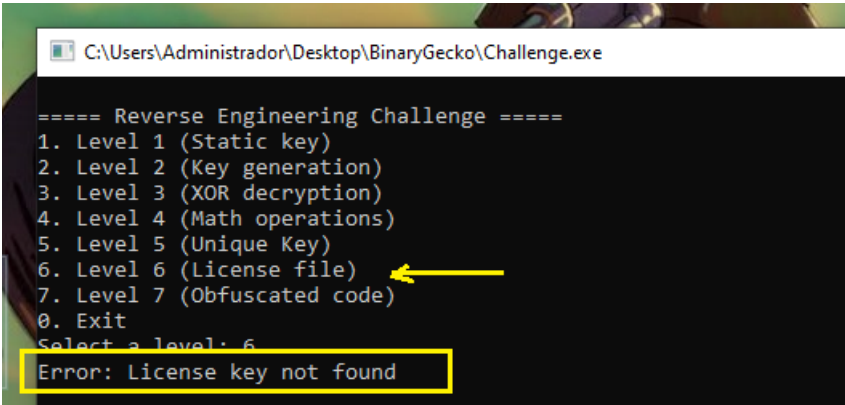


Now, since I also need to practice some programming, it occurred to me to make another Script (it's not really necessary, but I'll do it anyway).

It will be called Challenge5-BinaryGeckoENG.py and will be shipped along with this document and the other scripts.

Level 6 (License file)

Let's start the penultimate challenge, let's execute it first.



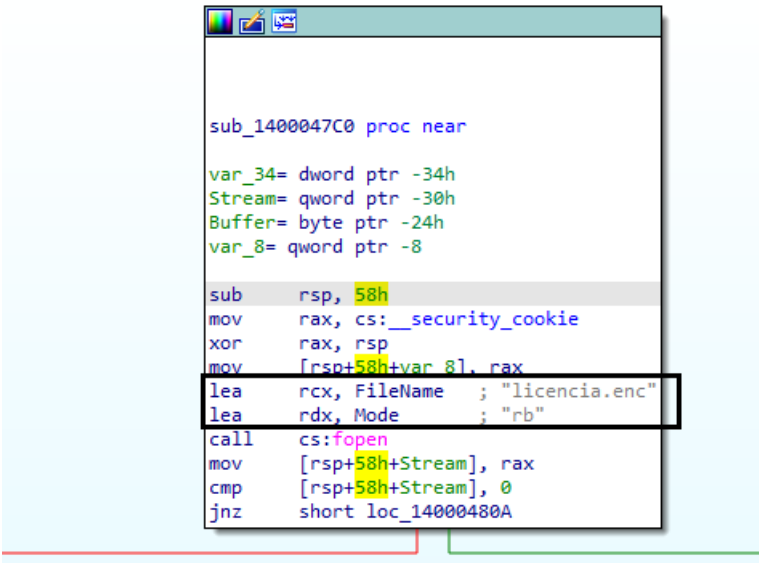
```
C:\Users\Administrador\Desktop\BinaryGecko\Challenge.exe

===== Reverse Engineering Challenge =====
1. Level 1 (Static key)
2. Level 2 (Key generation)
3. Level 3 (XOR decryption)
4. Level 4 (Math operations)
5. Level 5 (Unique Key)
6. Level 6 (License file)
7. Level 7 (Obfuscated code)
0. Exit
Select a level: 6
Error: License key not found
```

Ok, so as soon as I start it, it says “Error: License key not found ”

This means it is expecting a file, in my experience.

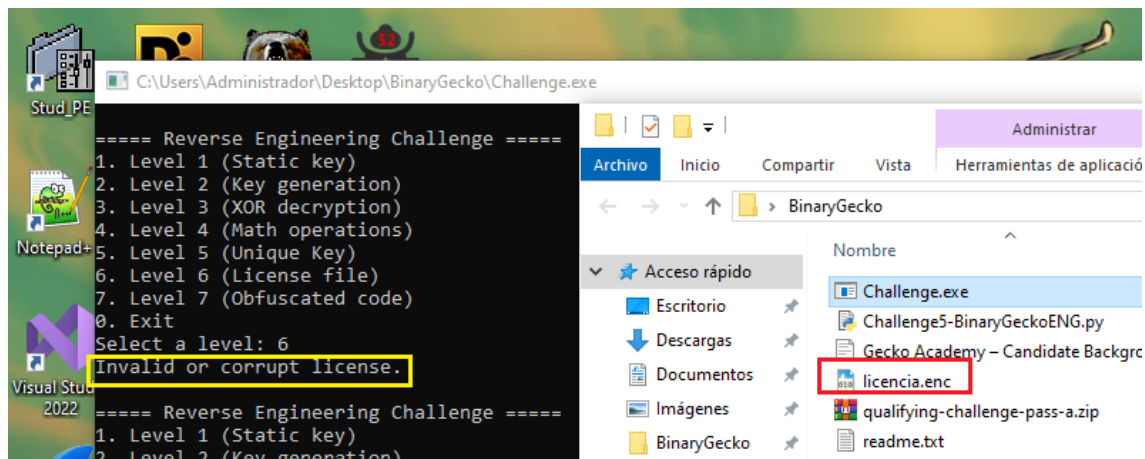
When opening in IDA Pro – it is very clear what I mentioned before



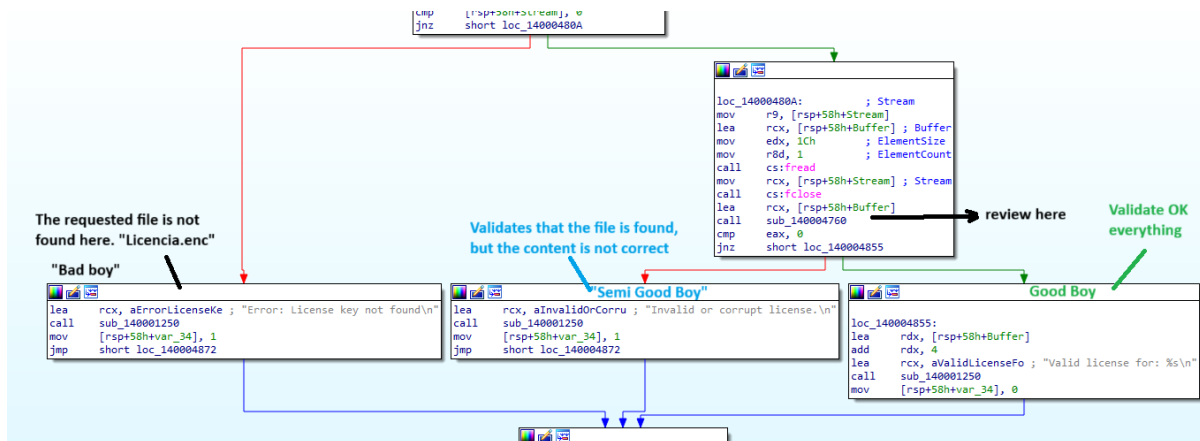
```
sub_1400047C0 proc near
var_34= dword ptr -34h
Stream= qword ptr -30h
Buffer= byte ptr -24h
var_8= qword ptr -8

sub     rsp, 58h
mov     rax, cs:__security_cookie
xor     rax, rsp
mov     [rsp+58h+var_8], rax
lea     rcx, FileName    ; "licencia.enc"
lea     rdx, Mode        ; "rb"
call    cs:fopen
mov     [rsp+58h+Stream], rax
cmp     [rsp+58h+Stream], 0
jnz     short loc_14000480A
```

licencia.enc ” file , but it also has permission to at least read it.



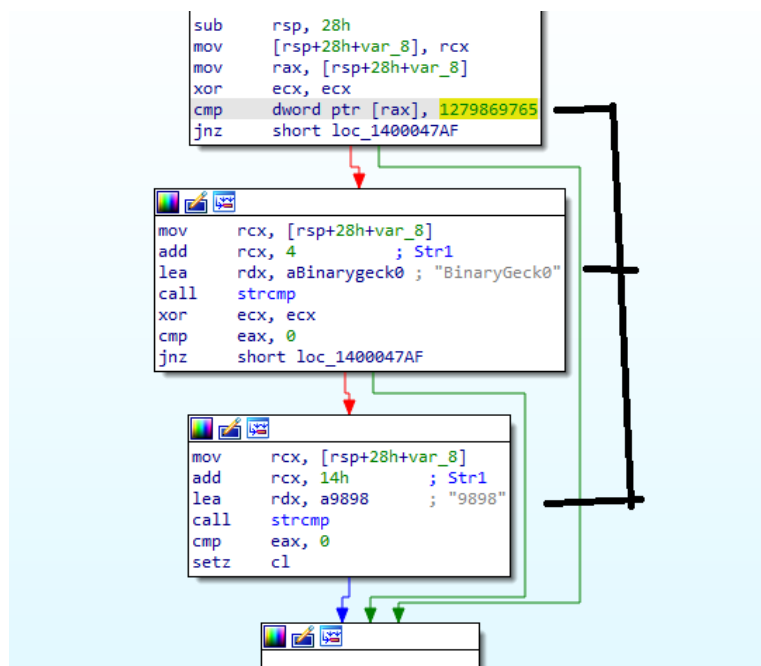
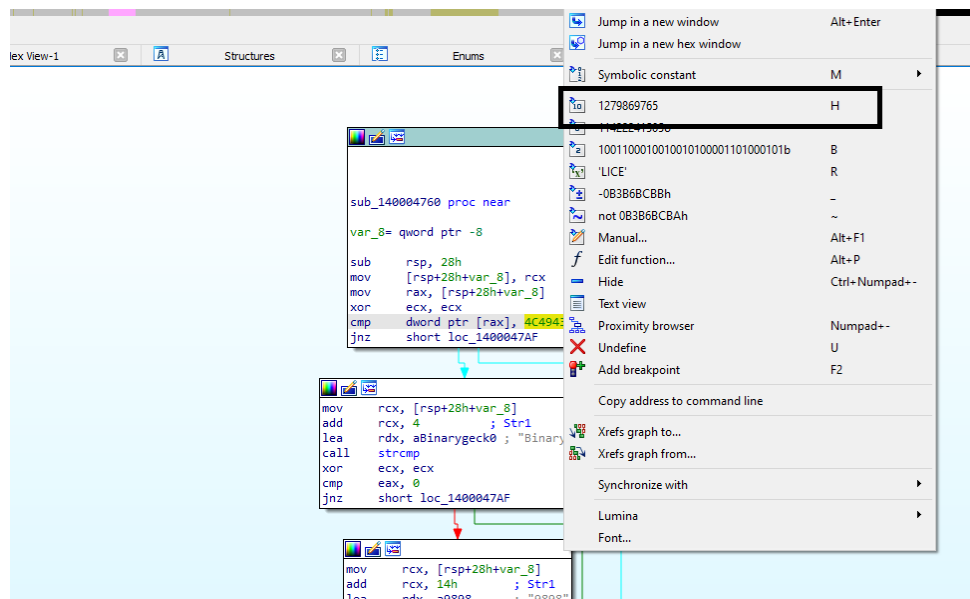
To validate my previous idea, I decided to create a file called " licencia.enc "—it's actually a notepad with that name—and I changed the extension. Now, we see that the message has changed, confirming both things: first, that it needs that file, and second, that it evaluates its contents.



In this image, in case it is not clear, sorry I thought it was smaller – we have a function to investigate, then 3 paths which I defined as “bad boy” which is when the licencia.enc file does not exist – Then there is “semi-good boy” – which validates that the file is there but also that the content is correct – and “good boy” which validates that the file and its content are OK.

Then in the pending function to review I find that it compares with the constant “1279869765” and it is not a minor piece of data, since in a sequence of ASCII characters it corresponds to the string “Lice”

This is because: 1279869765 in Big Endian → 4C,49,43,45 LICE // in Little Endian it would be ECIL (45 43 49 4C)

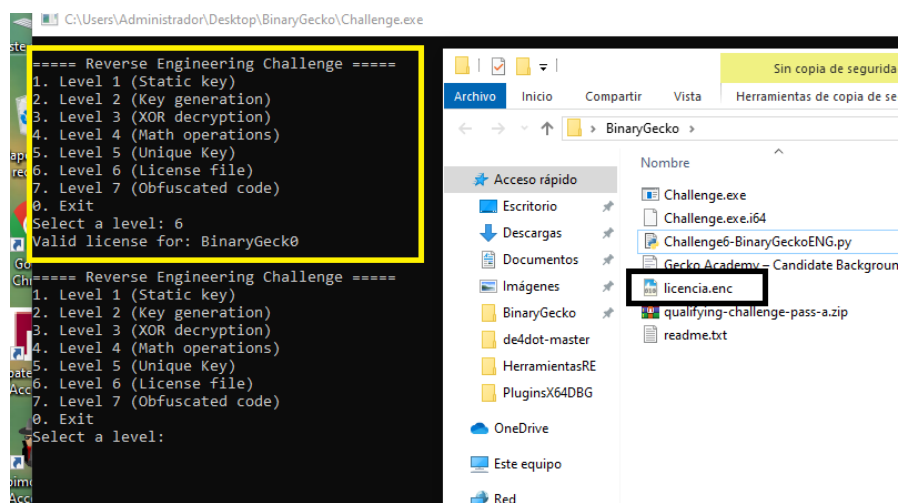


And then we see that there is the string BinaryGeck0 and 9898 – so, gathering all this information, it occurred to me that the file could contain the following strings “LICE” + “BinaryGeck0” + “9898” – I’ll be honest on this point too, the truth is that I tried several ways. And I always saved and tried different combinations and it never worked and I didn’t understand why, until I remembered a similar exercise, which instead of writing it manually in a document like a txt – I had to write it the same way in bytes.

I.e. , for example “ECIL+BinaryGeck0 +9898” – so I kept testing and it kept failing, until I realized that’s how I was writing it in the file. I finally realized that I should write it like this (I’m opening the Licencia.enc file with Notepad ++ and using the “HEX Editor” plugin)

:s	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	Dump
00	45	43	49	4c	42	69	6e	61	72	79	47	65	63	6b	30	ECILBinaryGeck0.
10	00	00	00	00	39	38	39	38	00							9898.

I needed to complete the sequence with 0 or something else so that it would take the key correctly (I would sincerely like to clear this up at some point because I didn't really understand the reason - I imagine it's because it needs to complete the number of bytes).

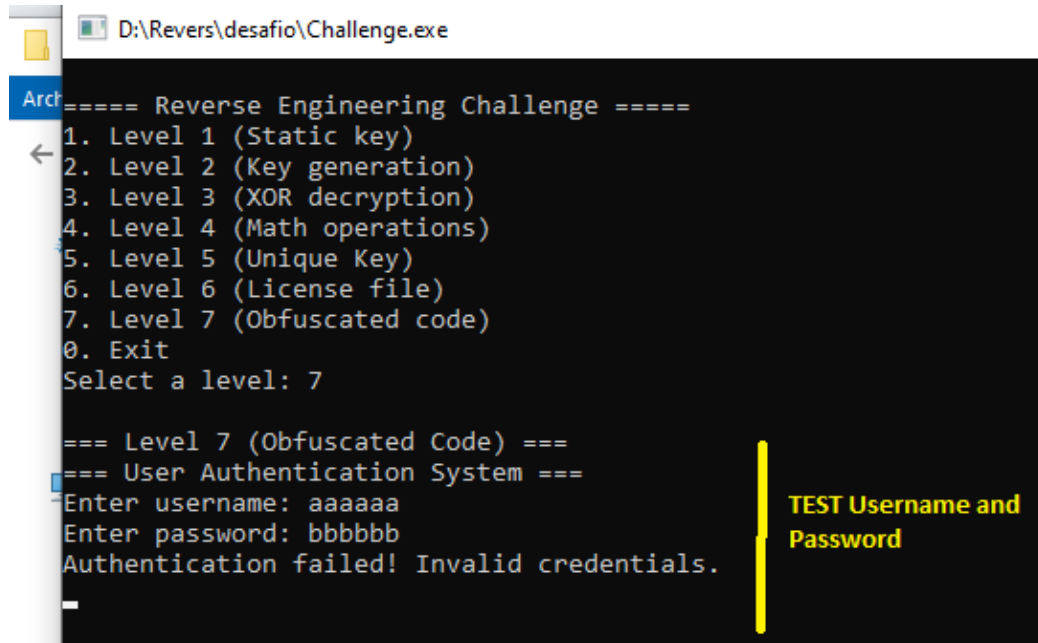


So, we can say that I finally found the solution, creating the file “licencia.enc” with the content already seen, when executing it it validates OK –

Of course, I generated a script (which, like the previous ones, will also be attached) with the name “Challenge6-BinaryGeckoENG.py”

Level 7 (Obsucated code)

When starting the last challenge, it directly asks me for a username and password, and since it is not correct, it tells us “ Authentication failed ! Invalid credentials . And then the executable ends.

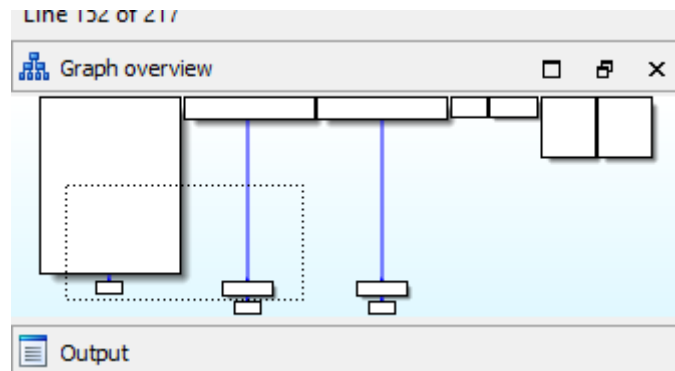


```
D:\Revers\desafio\Challenge.exe

===== Reverse Engineering Challenge =====
1. Level 1 (Static key)
2. Level 2 (Key generation)
3. Level 3 (XOR decryption)
4. Level 4 (Math operations)
5. Level 5 (Unique Key)
6. Level 6 (License file)
7. Level 7 (Obfuscated code)
0. Exit
Select a level: 7

=== Level 7 (Obfuscated Code) ===
=== User Authentication System ===
Enter username: aaaaaa
Enter password: bbbbbb
Authentication failed! Invalid credentials.
```

Indeed the code is obfuscated, we can notice it by some signs like this:



As proof, I open IDA Pro and see lines like the one marked – which indicate +”n” – in this case +1 – that is because it does not identify correctly.

```

sub_140001000 ; FUNCTION CHUNK AT .text:0000000140001220 SIZE 00000024 BYTES
sub_140001000
sub_140001000 ; unwind { // __CxxFrameHandler3
sub_140001000 push rbp
sub_140001000+1 push rsi
sub_140001000+2 push rdi
sub_140001000+3 push rbx
sub_140001000+4 sub rsp, 78h
sub_140001000+8 lea rbp, [rsp+70h]
sub_140001000+D mov [rbp+20h+var_20], 0FFFFFFFFFFFFFFEh
sub_140001000+15 lea rcx, aLevel70bfuscat ; "\n=== Level 7 (Obfuscated Code) ===\n"
sub_140001000+1C call sub_140001250
sub_140001000+21 mov [rbp+20h+var_24], 100h
sub_140001000+28 test ebp, ebp
sub_140001000+2A jnz short near ptr locret_14000102C+1

```

```

sub_140001000+2C
sub_140001000+2C locret_14000102C:
sub_140001000+2C retn 9090h

```

Here it clearly indicates that there is a C2 90 90 and in theory it should go to address 140001022D – but I can't find it here.

```

sub_140001000+21 43 00 00 01 00+ mov [rbp+20h+var_24], 100h
sub_140001000+28 85 ED test ebp, ebp
sub_140001000+2A 75 01 jnz short near ptr locret_14000102C+1
sub_140001000+2C
sub_140001000+2C locret_14000102C: ; CODE XREF: sub_140001000+2A1j
sub_140001000+2C C2 90 90 retn 9090h ; This takes 2 bytes
sub_140001000+2C
sub_140001000+2F 90 90 db 2 dup(90h)
sub_140001000+31 48 88 35 F8 5F 00+ db 48h, 88h, 35h, 0F8h, 5Fh, 2 dup(0)
sub_140001000+38 48 8D 15 09 54 00+ dq 4800005409158D48h, 4800000299E8F189h, 8B4C00000562158Dh
sub_140001000+38 00 48 89 F1 E8 99+ dq 0C1894800005FAB05h, 54FC158D48D0FF41h, 276E8F189480000h
sub_140001000+38 02 00 00 48 8D 15+ dq 8948D8758D480000h
sub_140001000+70 F1 E8 8A 05 00 00+ db 0F1h, 0E8h, 8Ah, 5, 2 dup(0), 48h
sub_140001000+77
sub_140001000+77 loc_140001077: ; DATA XREF: .rdata:0000000140008038lo
sub_140001000+77 ; try {
sub_140001000+77 8B 0D AB 5F 00 00 mov ecx, dword ptr cs:std:c:\n
sub_140001000+7D 48 89 F2 mov rdx, rsi
sub_140001000+80 E8 AB 05 00 00 call sub_140001630
sub_140001000+85 EB 00 jmp short $+2

```

I should find the address 140001022D - but I can't find it.

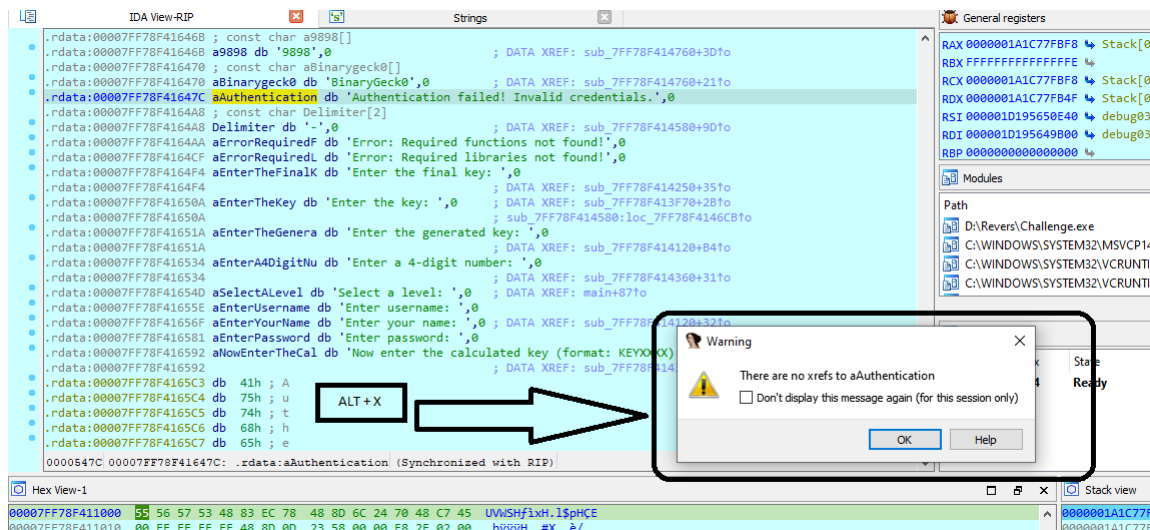
Then, we could go through the extensive code and rebuild everything that is allowed.

```

sub_140001000+8 48 8D 6C 24 70 lea rbp, [rsp+70h]
sub_140001000+D 48 C7 45 00 FE FF+ mov [rbp+20h+var_20], 0FFFFFFFFFFFFFFEh
sub_140001000+D FF FF
sub_140001000+15 48 8D 00 23 58 00+ lea rcx, aLevel70bfuscat ; "\n=== Level 7 (Obfuscated Code) ===\n"
sub_140001000+15 00
sub_140001000+1C E8 2F 02 00 00 call sub_140001250
sub_140001000+21 C7 45 FC 00 01 00+ mov [rbp+20h+var_24], 100h
sub_140001000+21 00
sub_140001000+28 85 ED test ebp, ebp
sub_140001000+28
sub_140001000+2A 75 01 db 75h ; u
sub_140001000+2B 01 db 1
sub_140001000+2C C2 db 0C2h ; A
sub_140001000+2D 90 db 90h
sub_140001000+2E 90 db 90h

```

Now 140001022D appears - this was done by going to instruction 140001022C and pressing the 'u' key

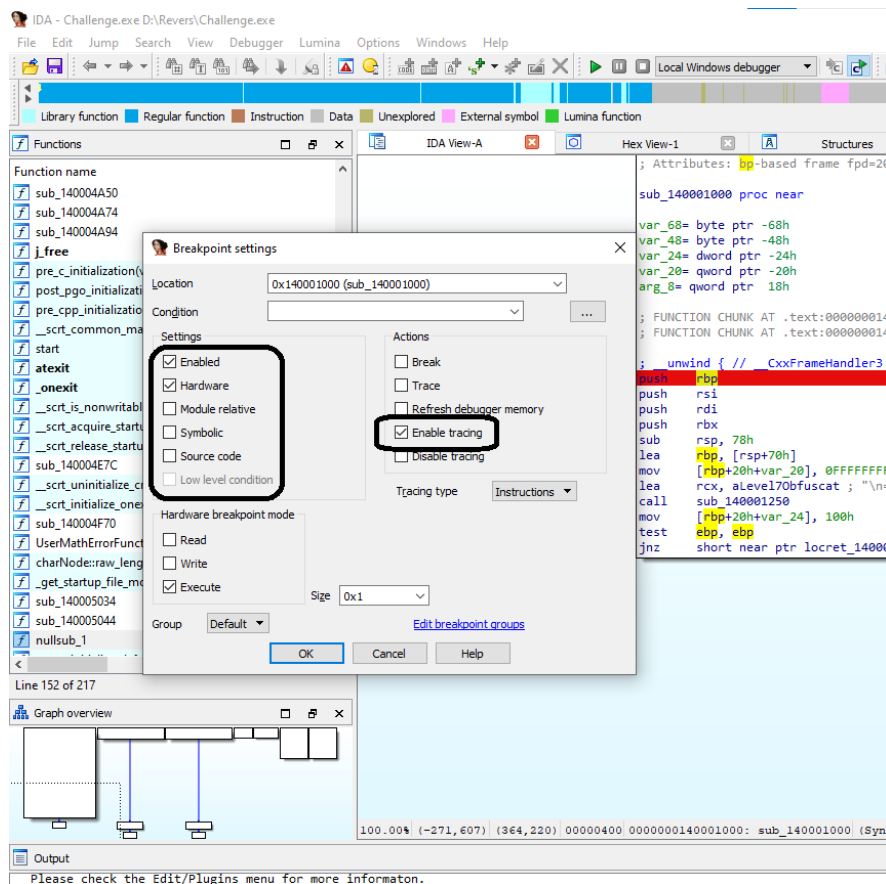


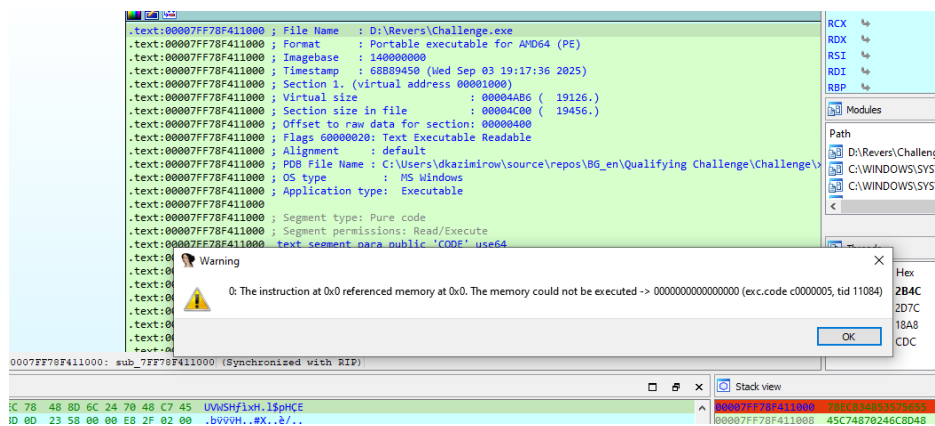
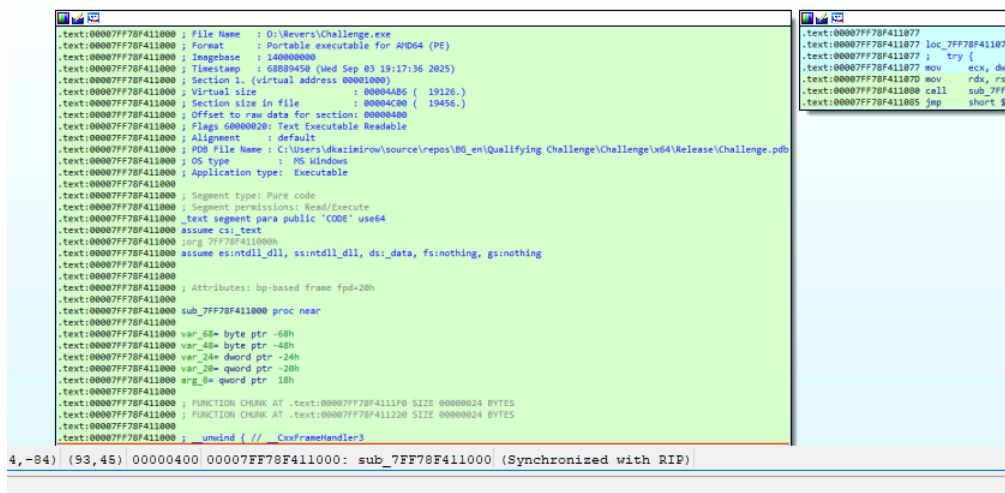
Here you give Alt + X for example to see the function of the string "Authentication" failed !..." and gives that error.

But I don't have much time and the truth is that someone with greater ability could generate a script if a pattern existed and remove the junk part and thus reconstruct the code, but that's not my case.

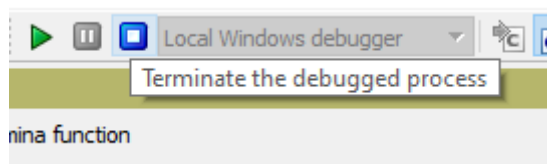
First I'm going to set a Hardware breakpoint – and enable "Enable" Tracing -

With this, I can temporarily "rebuild" it, and realize how to create the password :

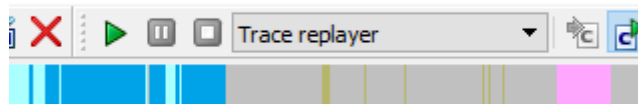




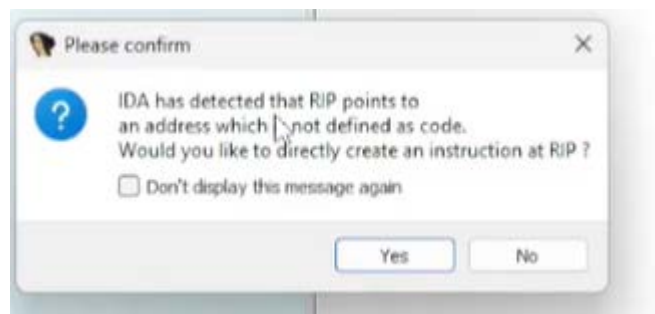
It notifies us that it has ended.



When we get to this point, we stop the Debugger and then select “Trace replayer”.

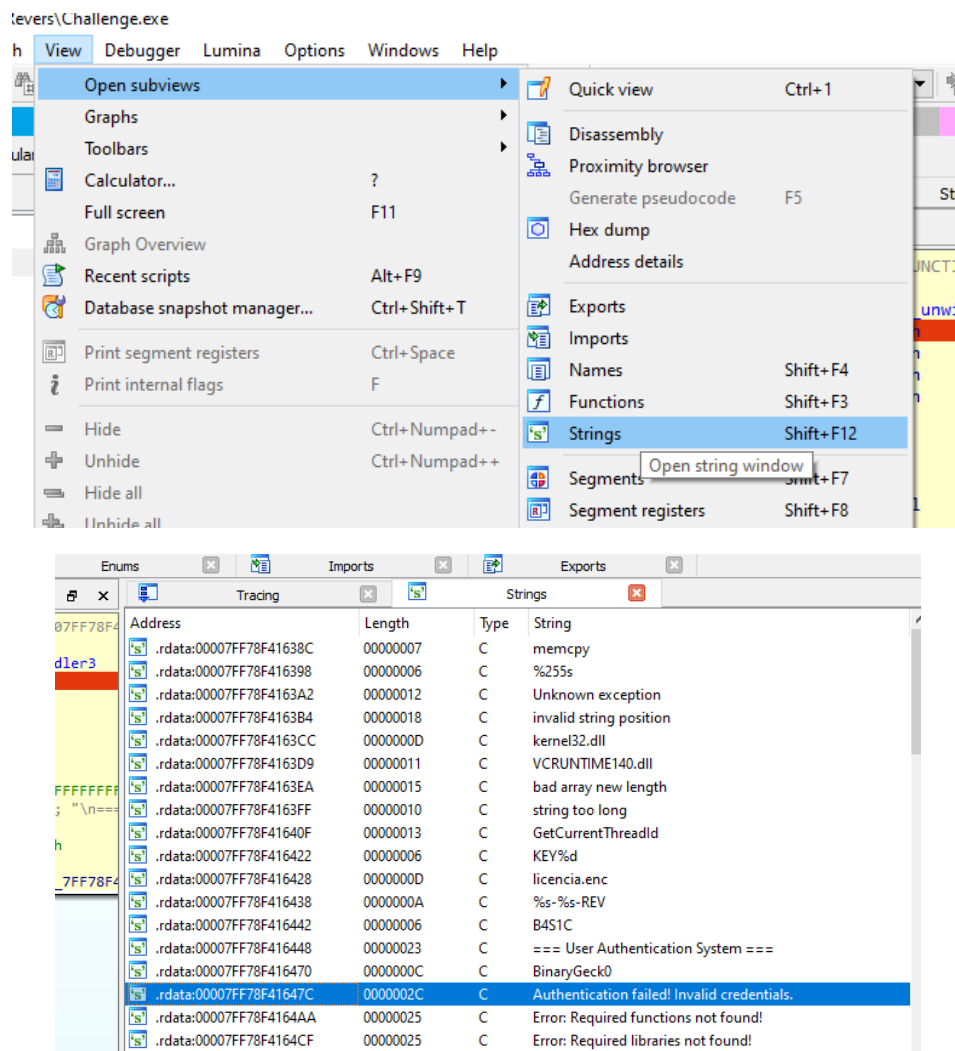


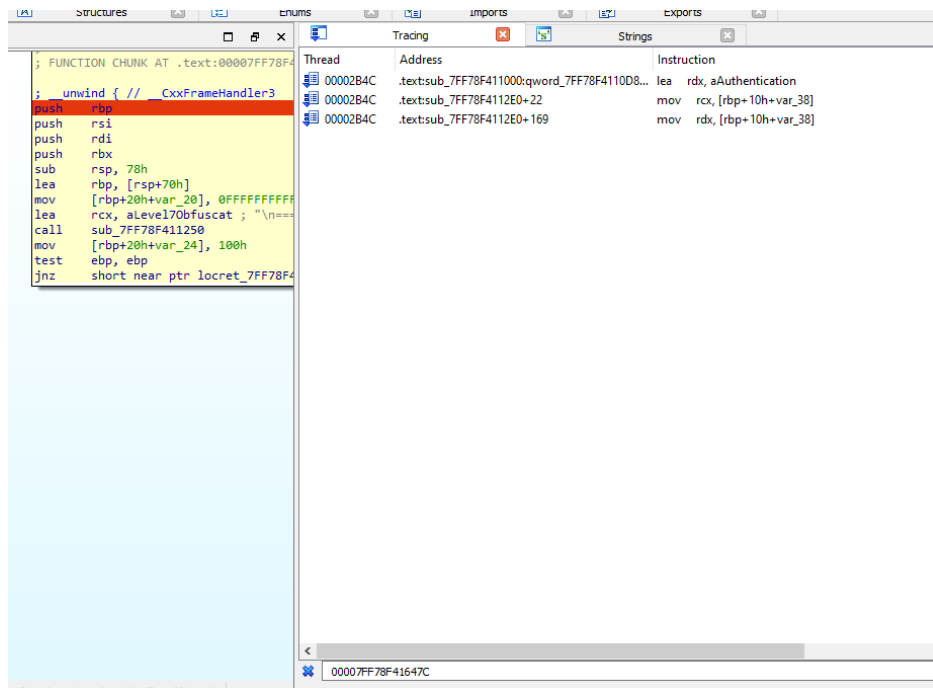
Surely when trying to view some function or something that is damaged this message will appear:



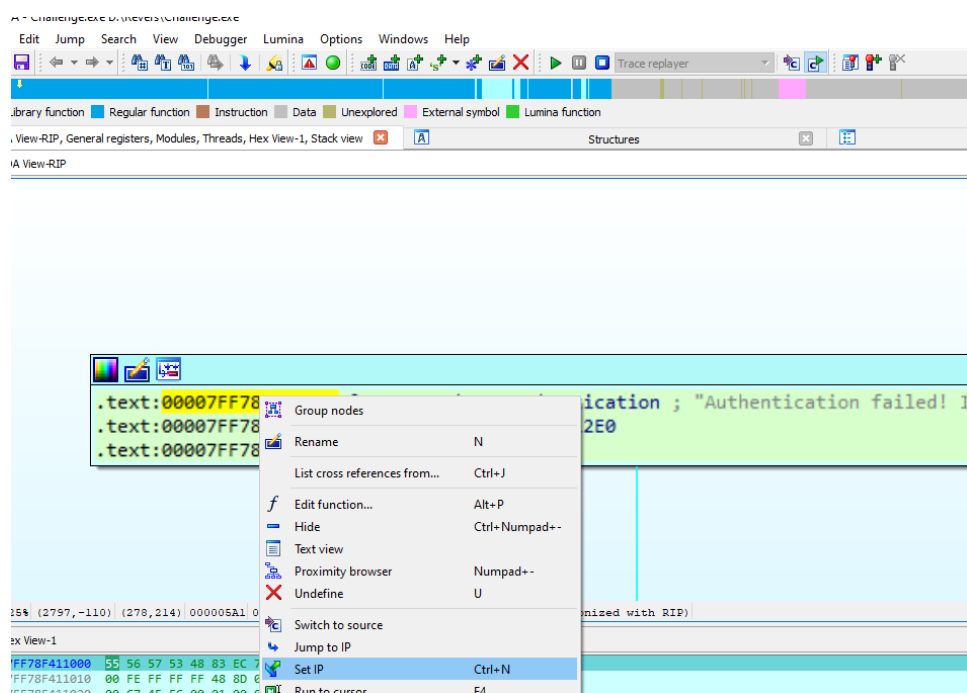
So as not to bother anyone, I checked it so it wouldn't show again and I put "Yes."

Then we look for the string :

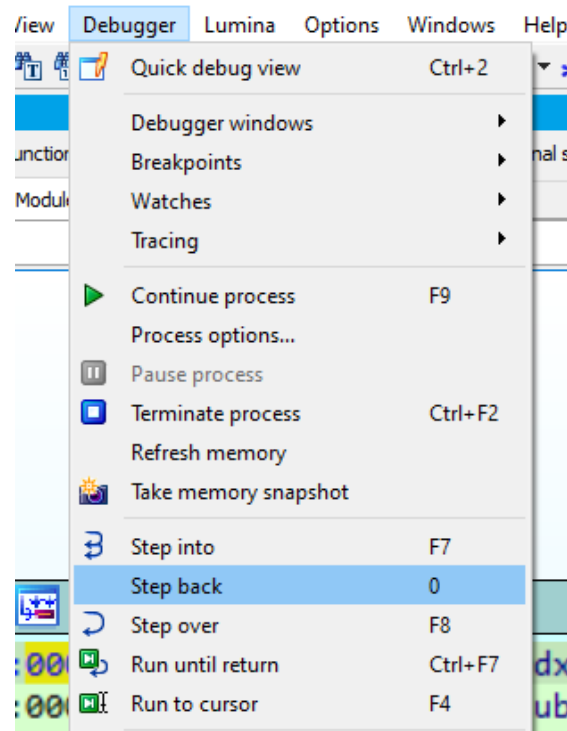




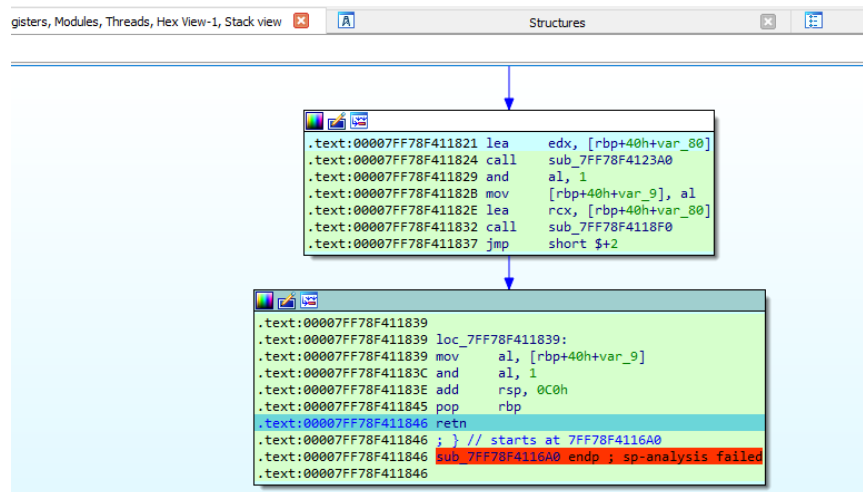
We see that its address is 00007FF6529F647C – so we look for it in the “Tracing” window.



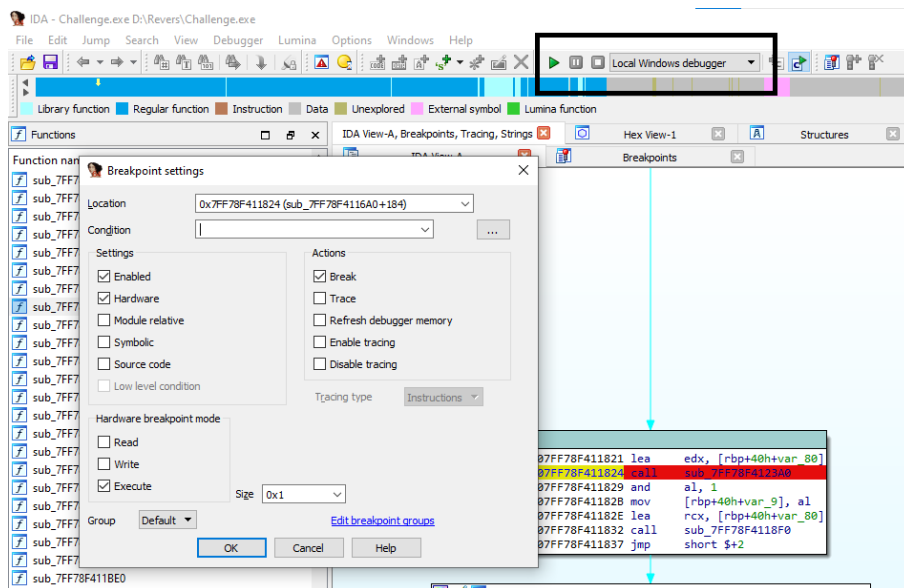
Remember to take note of the address where it is – in this case it was: 00007FF78F4111A1 (it varies) – and there we right click -> Set IP -



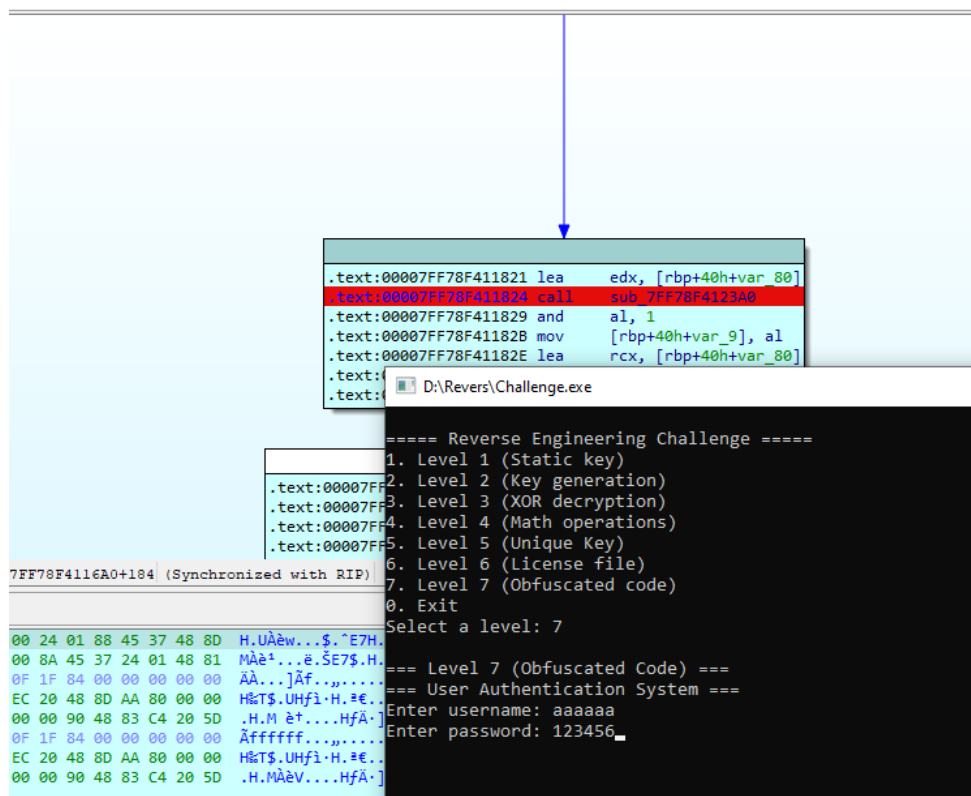
Then with Step back (in my case with number 0) we are going to reconstruct the section of the code that interests us.

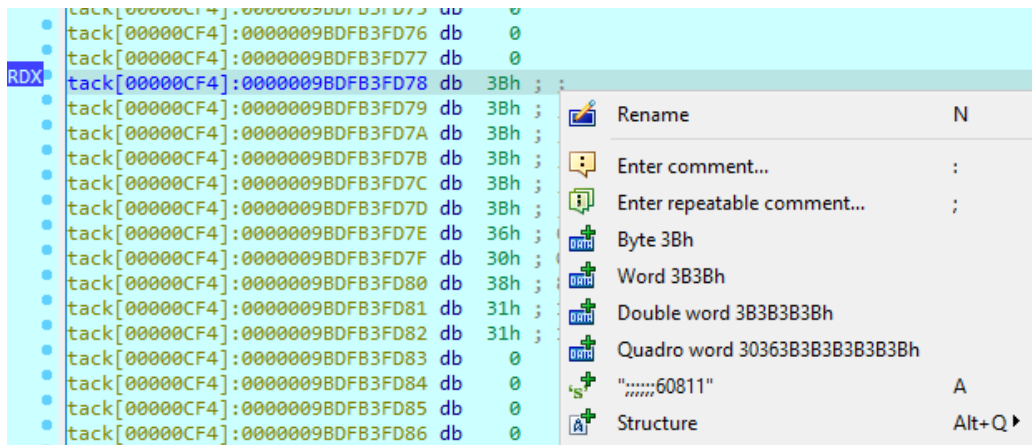


Then we set a hardware breakpoint – it is important to choose “Local Windows debugger ” again -

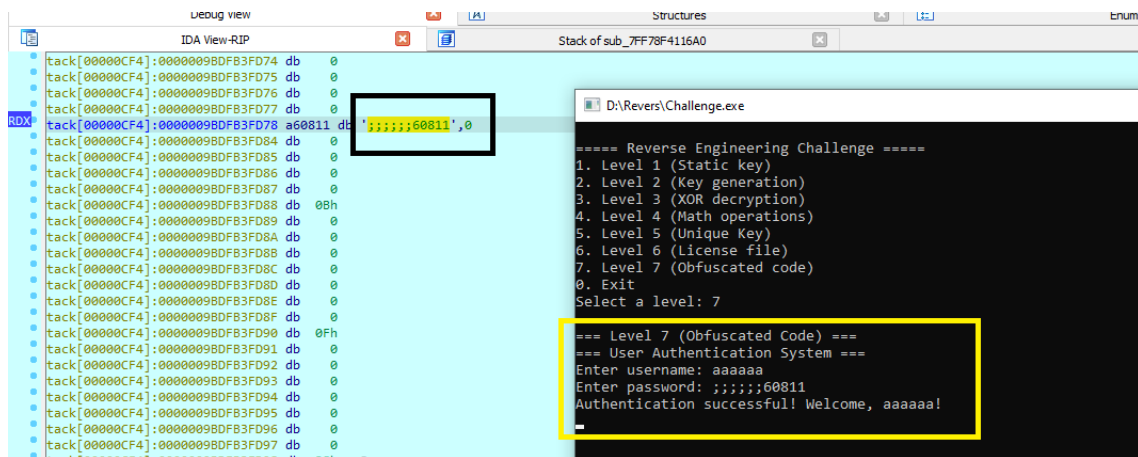


And we trace until we reach this call sub_7FF78F4123A0.



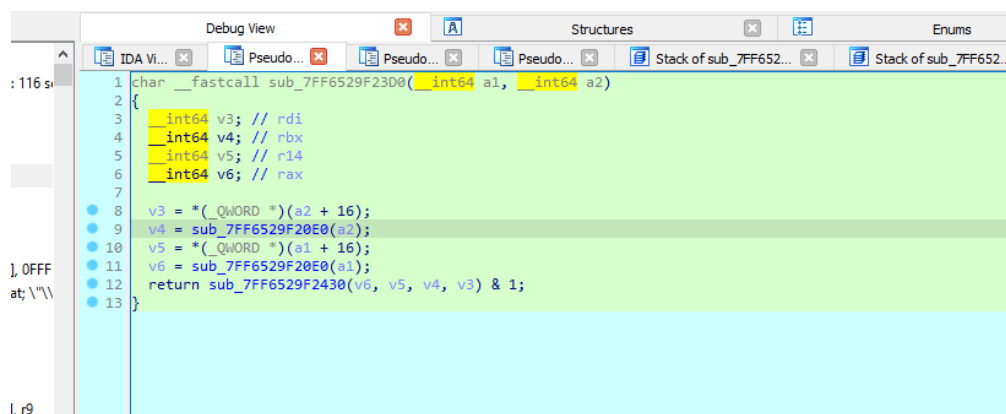


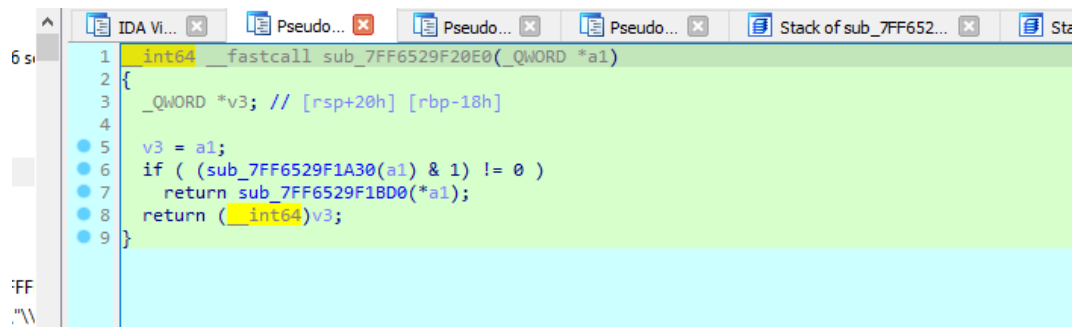
In the line that says “var_80” to see its contents, we right -click and select the corresponding option, or we type 'A' and create the string .



That would be the solution. Now, some personal details I'd like to share:

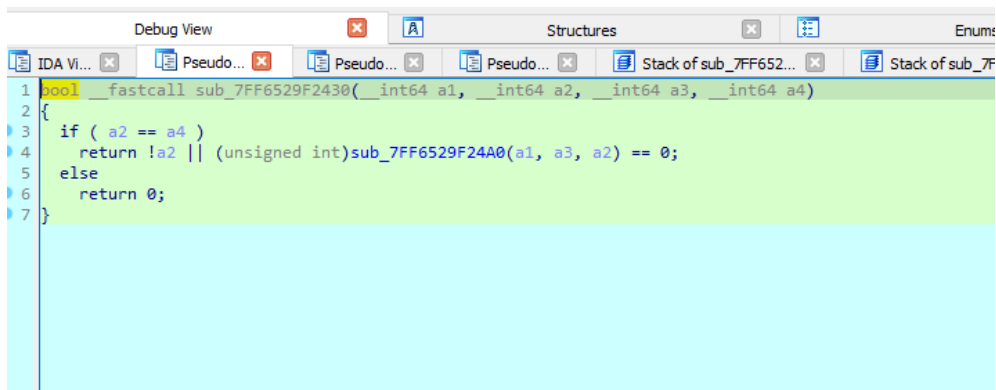
While I was tracing , it was actually quite complicated for me, with F5 – that is, with the pseudocode – I found some structures that caught my attention:





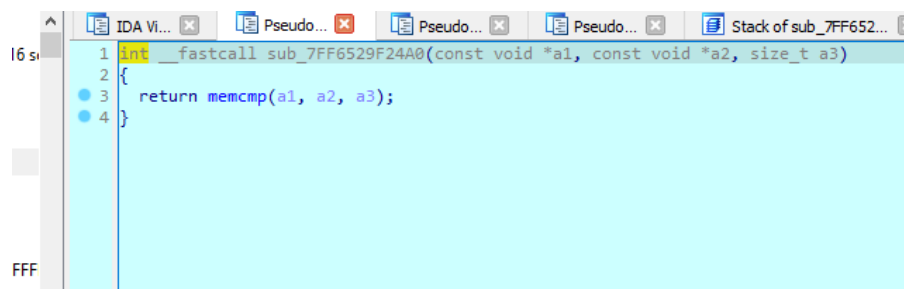
IDA Pro disassembly window showing a function call to `sub_7FF6529F20E0`. The code is as follows:

```
1  __int64 __fastcall sub_7FF6529F20E0(_QWORD *a1)
2  {
3      _QWORD v3; // [rsp+20h] [rbp-18h]
4
5      v3 = a1;
6      if ( (sub_7FF6529F1A30(a1) & 1) != 0 )
7          return sub_7FF6529F1BD0(*a1);
8      return (__int64)v3;
9  }
```



IDA Pro disassembly window showing a function call to `sub_7FF6529F2430`. The code is as follows:

```
1  bool __fastcall sub_7FF6529F2430(__int64 a1, __int64 a2, __int64 a3, __int64 a4)
2  {
3      if ( a2 == a4 )
4          return !a2 || (unsigned int)sub_7FF6529F24A0(a1, a3, a2) == 0;
5      else
6          return 0;
7  }
```

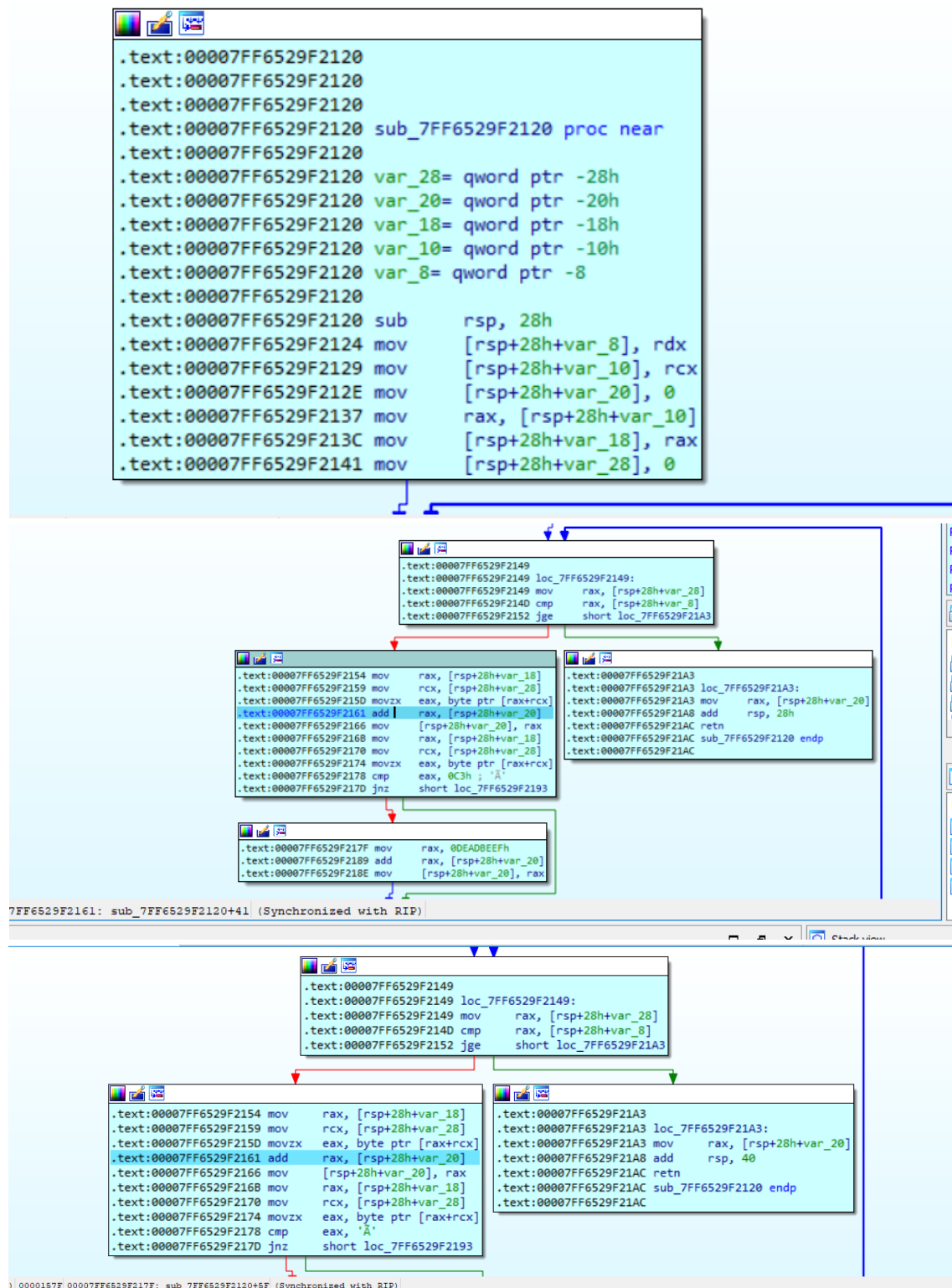


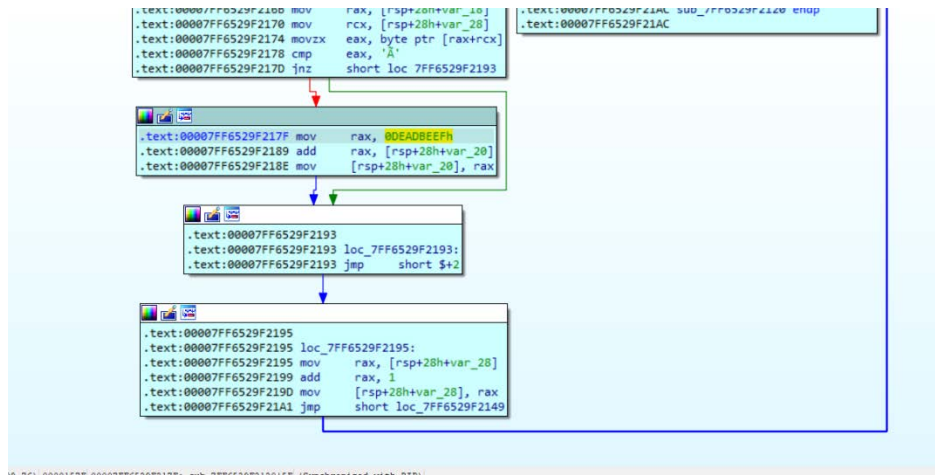
IDA Pro disassembly window showing a function call to `sub_7FF6529F24A0`. The code is as follows:

```
1  int __fastcall sub_7FF6529F24A0(const void *a1, const void *a2, size_t a3)
2  {
3      return memcmp(a1, a2, a3);
4  }
```

All this is just to confuse you, what matters is that it manages to perform the operation with `memcmp` .

Then, continuing with the tracing, I arrived at what I believe is the most important loop and where – although some details are missing – most of the calculations are performed:





Finally, I leave a screenshot of what the pseudocode would be according to IDA PRO:

```

1  int64 __fastcall sub_7FF6529F2120(__int64 a1, __int64 a2)
2  {
3      __int64 i; // [rsp+0h] [rbp-28h]
4      __int64 v4; // [rsp+8h] [rbp-20h]
5
6      v4 = 0i64;
7      for ( i = 0i64; i < a2; ++i )
8      {
9          v4 += *(unsigned __int8 *)(a1 + i);
10         if ( *(unsigned __int8 *)(a1 + i) == '\xC3' )
11             v4 += 3735928559i64;
12     }
13     return v4;
14 }

```

Final conclusion

As a final note, I didn't want to miss the opportunity to thank you for the challenges. They were really fun, testing my knowledge and, at times, my patience. I think I can say that on a personal level, I feel satisfied. When I started, I thought I wouldn't even be able to solve the first one, but to my surprise, I was able to complete all of them.

Knowledge from different tools such as IDA PRO (which I had used for some things up to now but not at the level I required for this series of challenges) was applied.

I was able to gain more confidence in developing scripts (which I don't usually do for various reasons). Each of the challenges made me think differently, but it was also a reflection of the current knowledge I currently have, and that I require more knowledge to continue advancing in the world of reverse engineering.

Scripts

Challenge2-BinaryGeckoENG.py

```
Challenge2-BinaryGeckoENG.py x Ejercicio16.txt
AprendiendoReversing > Challenge2-BinaryGeckoENG.py > ...
1 user_name = input("Insert your name: ")
2 intermediate_key = []
3 for char in user_name:
4     new_char_code = ord(char) + 3
5     new_char = chr(new_char_code)
6     intermediate_key.append(new_char)
7 ciphered_string = "".join(intermediate_key)
8 final_key = ciphered_string[::-1]
9 print("\n--- Result ---")
10 print(f"Usuario: {user_name}")
11 print(f"KeyGen: {final_key}")
12 print("-----")

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS

PS C:\EntornoDesarrollo\MisProgramas\AprendiendoReversing> & C:/EntornoDesarrollo/MisProgramas/AprendiendoReversing/Challenge2-BinaryGeckoENG.py
Insert your name: solid

--- Result ---
Usuario: solid
KeyGen: glorv
-----

PS C:\EntornoDesarrollo\MisProgramas\AprendiendoReversing>
```

Note: This is actually the second version I made, because the first one didn't take case sensitivity into account - if the name said " Karasu " and provided the wrong key, so that was fixed:

I leave some tests carried out:

```
rollo/MisProgramas/Aprendiendo
Insert your name: Ricardo

--- Result ---
Usuario: Ricardo
KeyGen: rgudflU
-----
```

```
exit
=== Level 2 ===
Goal: Get the correct key for your name. (Optional write a keygen)
Enter your name: Ricardo
Enter the generated key: rgudflU
[+] Correct! Generated key: rgudflU
```

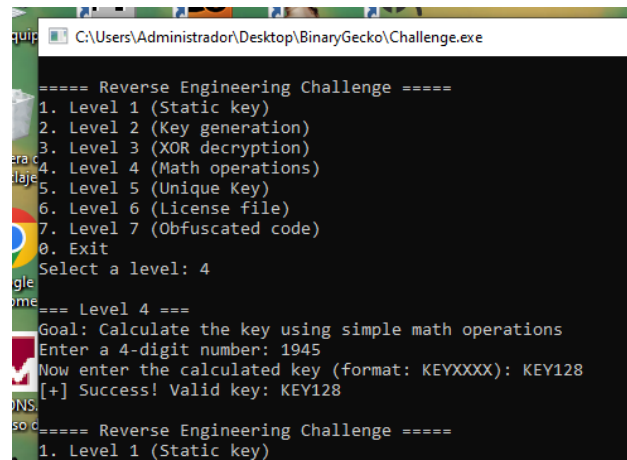
```
rollo/MisProgramas/Aprendiendo
Insert your name: SoLiD

--- Result ---
Usuario: SoLiD
KeyGen: GLoRV
```

```
rt
=== Level 2 ===
Goal: Get the correct key for your name. (Optional write a keygen)
rt Enter your name: SoLiD
rt Enter the generated key: GLoRV
rt [+] Correct! Generated key: GLoRV
```

Challenge4-BinaryGeckoENG .py

```
1 valid_entry = False
2 while not valid_entry:
3     entrynum = input("Enter a 4-digit number (ej: 1933): ")
4     if len(entrynum) == 4 and entrynum.isdigit():
5         valid_entry = True
6     else:
7         print("Error: Please enter exactly 4 digits. Please try again.")
8 pair1_str = entrynum[0:2]
9 pair2_str = entrynum[2:4]
10 pair1 = int(pair1_str)
11 pair2 = int(pair2_str)
12 sum = pair1 + pair2
13 keyfinal = sum * 2
14 keyfinal = "KEY" + str(keyfinal)
15 print("\n--- CALCULATION DETAILS ---")
16 print(f"Number entered: {entrynum}")
17 print(f"  - First Pair: {pair1}")
18 print(f"  - Second Pair: {pair2}")
19 print(f"  - Addition: {pair1} + {pair2} = {sum}")
20 print(f"  - Result: {sum} x 2 = {keyfinal}")
21 print(f"\nThe generated key is: {keyfinal}")
```



```
C:\Users\Administrador\Desktop\BinaryGecko\Challenge.exe

===== Reverse Engineering Challenge =====
1. Level 1 (Static key)
2. Level 2 (Key generation)
3. Level 3 (XOR decryption)
4. Level 4 (Math operations)
5. Level 5 (Unique Key)
6. Level 6 (License file)
7. Level 7 (Obfuscated code)
0. Exit
Select a level: 4

===== Level 4 =====
Goal: Calculate the key using simple math operations
Enter a 4-digit number: 1945
Now enter the calculated key (format: KEYXXXX): KEY128
[+] Success! Valid key: KEY128

===== Reverse Engineering Challenge =====
1. Level 1 (Static key)
```

```

Challenge2-BinaryGeckoENG.py X Challenge4-BinaryGeckoENG.py O X
AprendiendoReversing > Challenge4-BinaryGeckoENG.py > ...
6         else:
7             print("Error: Please enter exactly 4 digits. Plea
8
9         pair1_str = entrynum[0:2]
10        pair2_str = entrynum[2:4]
11        pair1 = int(pair1_str)
12        pair2 = int(pair2_str)
13        sum = pair1 + pair2
14        keyfinal = sum * 2
15        keyfinal = "KEY" + str(keyfinal)
16
17        print("\n--- CALCULATION DETAILS ---")
18        print(f"Number entered: {entrynum}")
19        print(f" - First Pair: {pair1}")
20        print(f" - Second Pair: {pair2}")
21        print(f" - Addition: {pair1} + {pair2} = {sum}")
22        print(f" - Result: {sum} x 2 = {keyfinal}")
23        print(f"\nThe generated key is: {keyfinal}")

```

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS

```

PS C:\EntornoDesarrollo\MisProgramas> & C:/EntornoDesarrollo/Python
o/MisProgramas/AprendiendoReversing/Challenge4-BinaryGeckoENG.py
Enter a 4-digit number (ej: 1933): 1945

--- CALCULATION DETAILS ---
Number entered: 1945
- First Pair: 19
- Second Pair: 45
- Addition: 19 + 45 = 64
- Result: 64 x 2 = KEY128

The generated key is: KEY128

```

Other tests performed

```

16 print(f"Number entered: {entrynum}")
17 print(f" - First Pair: {pair1}")
18 print(f" - Second Pair: {pair2}")
19 print(f" - Addition: {pair1} + {pair2} = {sum}")
20 print(f" - Result: {sum} x 2 = {keyfinal}")
21 print(f"\nThe generated key is: {keyfinal}")

```

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS

```

PS C:\EntornoDesarrollo\MisProgramas> & C:/EntornoDesarrollo
o/MisProgramas/AprendiendoReversing/Challenge4-BinaryGeckoENG.py
Enter a 4-digit number (ej: 1933): 9898

--- CALCULATION DETAILS ---
Number entered: 9898
- First Pair: 98
- Second Pair: 98
- Addition: 98 + 98 = 196
- Result: 196 x 2 = KEY392

The generated key is: KEY392
PS C:\EntornoDesarrollo\MisProgramas>

```

```

C:\Users\Administrador\Desktop\BinaryGecko\Challenge.exe
Now enter the calculated key (format: KEYXXXX): KEY128
[+] Success! Valid key: KEY128

===== Reverse Engineering Challenge =====
1. Level 1 (Static key)
2. Level 2 (Key generation)
3. Level 3 (XOR decryption)
4. Level 4 (Math operations)
5. Level 5 (Unique Key)
6. Level 6 (License file)
7. Level 7 (Obfuscated code)
0. Exit
Select a level: 4

=== Level 4 ===
Goal: Calculate the key using simple math operations
Enter a 4-digit number: 9898
Now enter the calculated key (format: KEYXXXX): KEY392
[+] Success! Valid key: KEY392

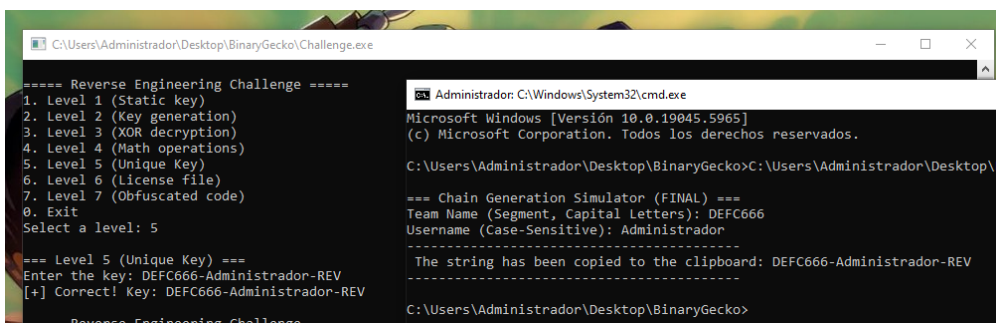
===== Reverse Engineering Challenge =====
1. Level 1 (Static key)
2. Level 2 (Key generation)
3. Level 3 (XOR decryption)
4. Level 4 (Math operations)
5. Level 5 (Unique Key)
6. Level 6 (License file)
7. Level 7 (Obfuscated code)
0. Exit
Select a level:

```

Challenge5-BinaryGeckoENG.py

```
Challenge2-BinaryGeckoENG.py Challenge4-BinaryGeckoENG.py Challenge5-BinaryGeckoENG.py U X
AprendiendoReversing > Challenge5-BinaryGeckoENG.py > ...
1 import socket
2 import getpass
3 import pyperclip
4
5 namepccomplete = socket.gethostname()
6
7 if '-' in namepccomplete:
8     namesegment = namepccomplete.split('-')[1]
9 else:
10    namesegment = namepccomplete
11    namesegment = namesegment.upper()
12 username = getpass.getuser()
13 basestrings = namesegment + "-" + username + "-REV"
14 try:
15     pyperclip.copy(basestrings)
16     print("\n=== Chain Generation Simulator (FINAL) ===")
17     print(f"Team Name (Segment, Capital Letters): {namesegment}")
18     print(f"Username (Case-Sensitive): {username}")
19     print("-----")
20     print(f"The string has been copied to the clipboard: {basestrings}")
21     print("-----")
22 except pyperclip.PyperclipException as e:
23
24     print("\nERROR: Could not access clipboard.")
25     print("Make sure you have a graphical environment or the necessary dependencies (e.g. xclip/xsel on Linux).")
26     print(f"The generated string is: {basestrings}")
27     print(f"Error details: {e}")
```

I know that whoever reads this document knows infinitely more than I do, but just in case I clarify that for the script to work you must have pyperclip installed – From CMD (Windows) -> pip install pyperclip (this is so that when I generate the name of the PC and concatenate it and create the corresponding key, it remains in the clipboard.



```
C:\Users\Administrador\Desktop\BinaryGecko\Challenge.exe
===== Reverse Engineering Challenge =====
1. Level 1 (Static key)
2. Level 2 (Key generation)
3. Level 3 (XOR decryption)
4. Level 4 (Math operations)
5. Level 5 (Unique Key)
6. Level 6 (License file)
7. Level 7 (Obfuscated code)
0. Exit
Select a level: 5

=== Level 5 (Unique Key) ===
Enter the key: DEFC666-Administrador-REV
[+] Correct! Key: DEFC666-Administrador-REV
===== Reverse Engineering Challenge =====

C:\Windows\System32\cmd.exe
Microsoft Windows [Versión 10.0.19045.5965]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Administrador\Desktop\BinaryGecko>C:\Users\Administrador\Desktop\B

=== Chain Generation Simulator (FINAL) ===
Team Name (Segment, Capital Letters): DEFC666
Username (Case-Sensitive): Administrador
-----
The string has been copied to the clipboard: DEFC666-Administrador-REV
-----

C:\Users\Administrador\Desktop\BinaryGecko>
```

Challenge6-BinaryGeckoENG.py

```
import os
import struct
import sys
destination_directory = r"C:\Users\Administrador\Desktop\BinaryGecko"
filename = "licencia.enc"
pathcomplete = os.path.join(destination_directory, filename)
head = struct.pack("<I", 1279869765)
substring1 = b"BinaryGecko\x00"
stuffed = b"\x00\x00\x00\x00"
substring2 = b"9898\x00"
final = head + substring1 + stuffed + substring2

try:
    os.makedirs(destination_directory, exist_ok=True)
    with open(pathcomplete, "wb") as f:
        f.write(final)

    print(f"    Success! File '{filename}' generated.")
    print(f"    Size: {len(final)} bytes")
    print(f"    Save path: {pathcomplete}")

except PermissionError:
    print(f"    Permissions error: Could not write to path '{destination_directory}'.", file=sys.stderr)
    print("    Make sure you run the script with sufficient permissions.", file=sys.stderr)
except Exception as e:
    print(f"    Unexpected error while writing file: {e}", file=sys.stderr)
```